Submission Template for

# INFS2200/INFS7903 Assignment 2

Semester 2, 2026

## Submission of Screenshot for Assignment 2

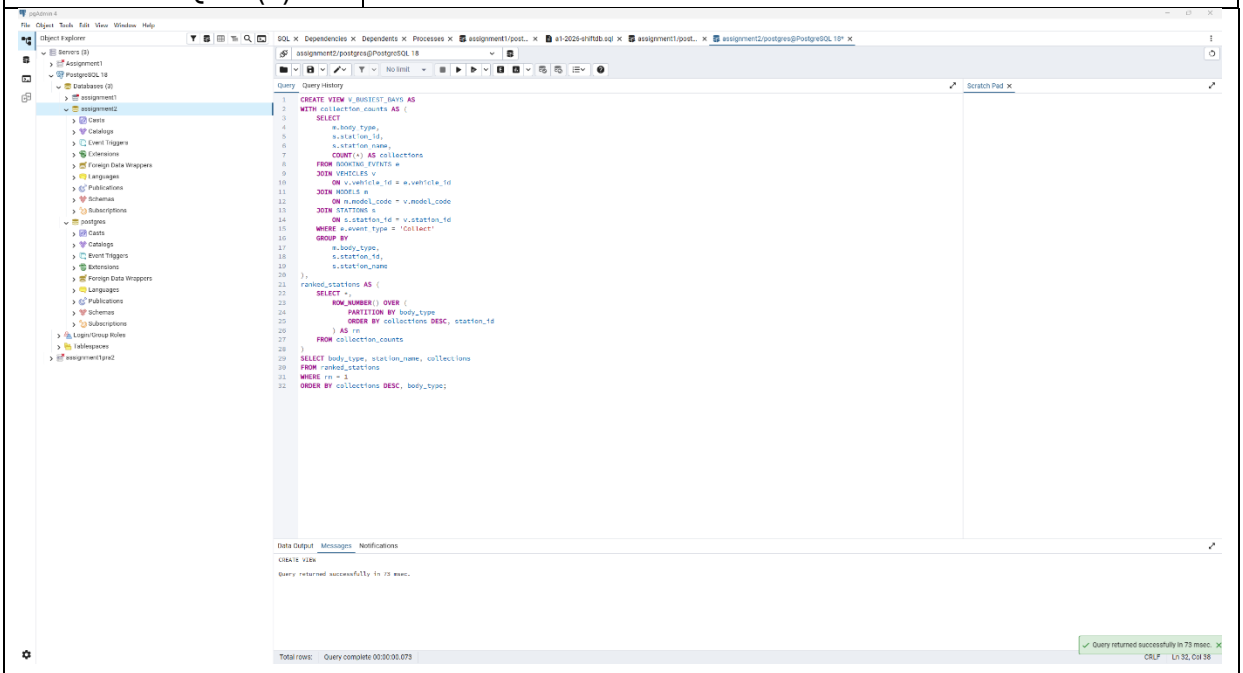
**Submitted by**:

| Student Name | Student ID | Practical Session | Tutor Name |
|---|---|---|---|
| Yuyang Gong | 49079984 | Friday 9:00–10:00 | Mr. Li, Jiayun |

About AI Usage:

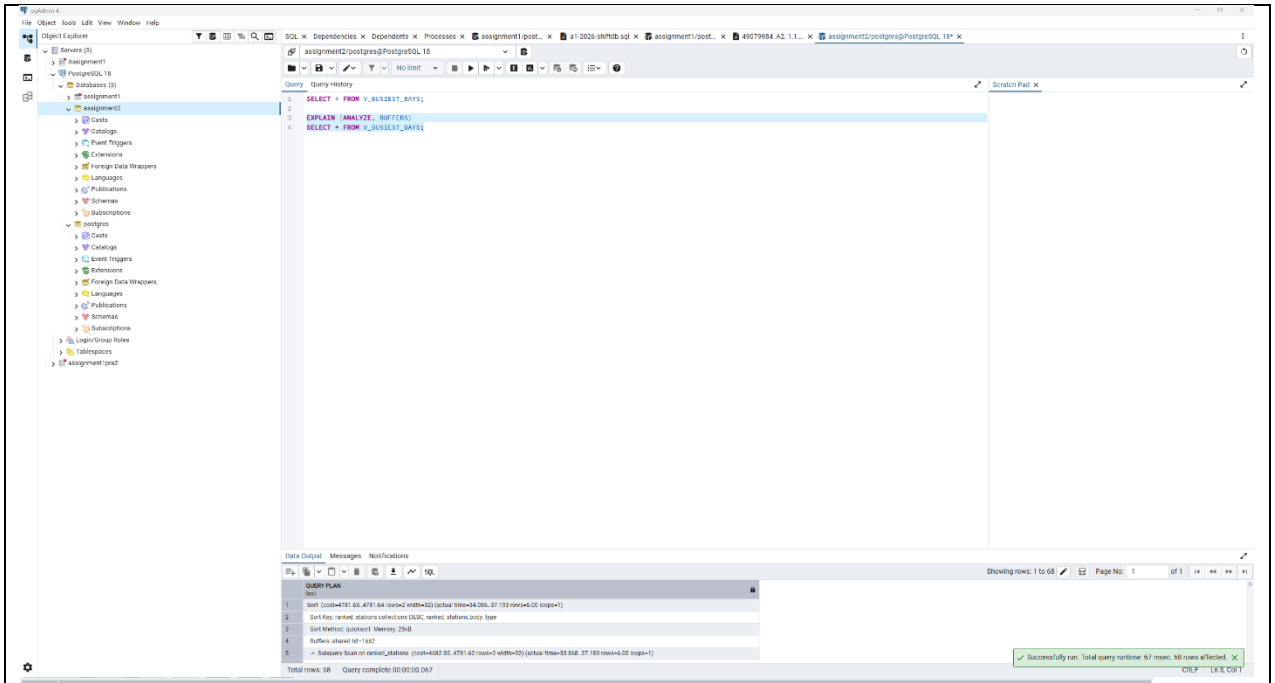
| | |
|---|---|
| Did you use AI in this assignment? | Yes |
| The purpose of AI used in this assignment? | Check my code and help me understand the assignment requirements. |
| To what extend that AI is used in this submitted work? | The code for each question has been checked. |
| Did you discuss your AI usage with the tutor during the "*Show to Tutor*" Lab sessions? | No. A2 has not yet been shown to the tutor, and this AI use has not yet been discussed. |

| Screenshot Q1.1 (1) | Marks: /1 |
|---|---|




Comments:

| Screenshot Q1.1(2) | Marks: /0.5 |
|---|---|

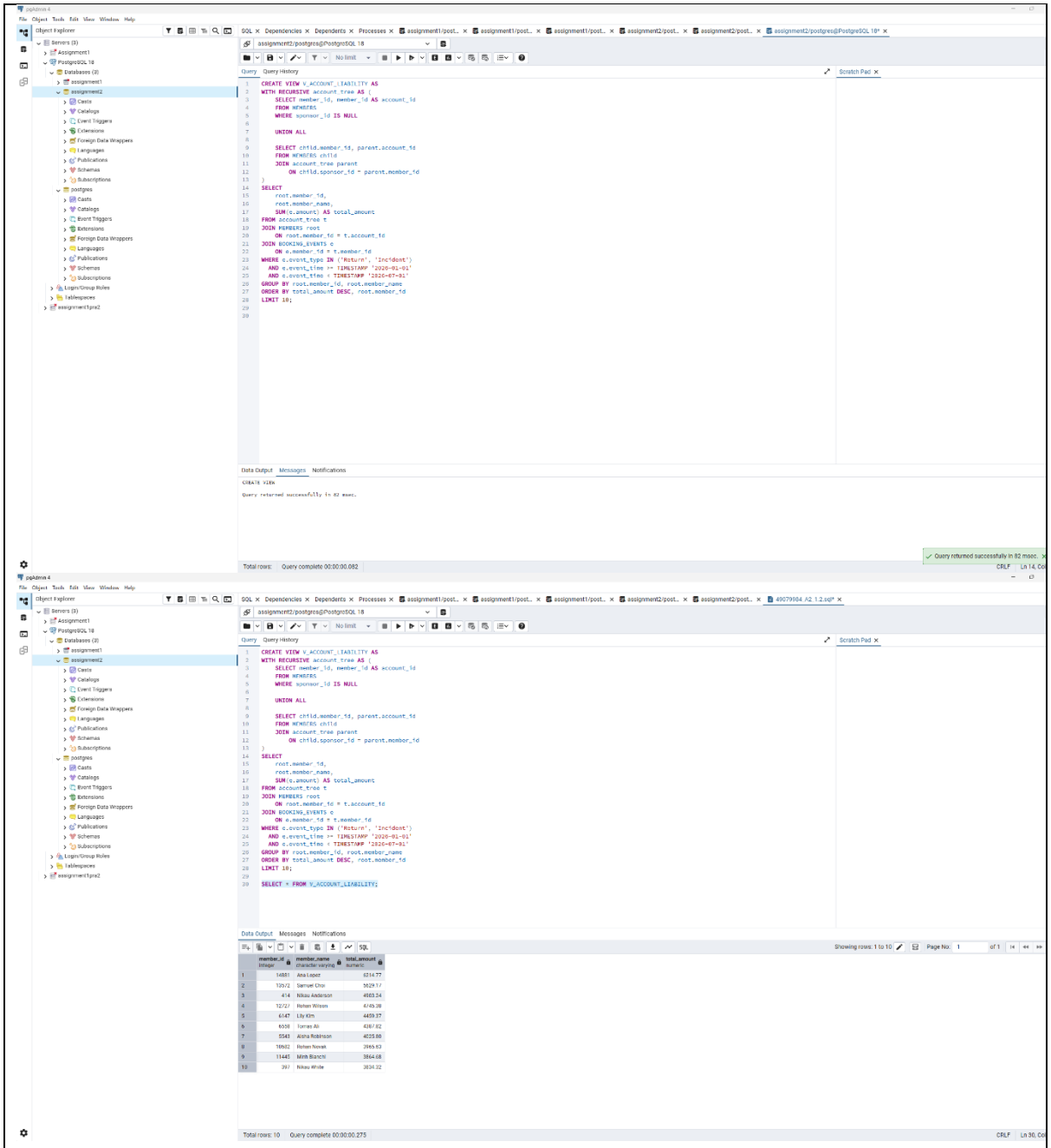



| | QUERY PLAN text |
|---|---|
| 1 | Sort (cost=4786.08..4786.08 rows=2 width=32) (actual time=47.879..52.128 rows=6.00 loops=1) |
| 2 | Sort Key: ranked_stations.collections DESC, ranked_stations.body_type |
| 3 | Sort Method: quicksort Memory: 25kB |
| 4 | Buffers: shared hit=1667 |
| 5 | -> Subquery Scan on ranked_stations (cost=4686.49..4786.07 rows=2 width=32) (actual time=47.652..52.118 rows=6.00 loops=1) |
| 6 | Filter: (ranked_stations.rn = 1) |
| 7 | Buffers: shared hit=1664 |
| 8 | -> WindowAgg (cost=4686.49..4781.57 rows=360 width=47) (actual time=47.651..52.116 rows=6.00 loops=1) |
| 9 | Window: w1 AS (PARTITION BY collection_counts.body_type ORDER BY collection_counts.collections, collection_counts.station_id ROWS UNBOUNDED PRECEDING) |
| 10 | Run Condition: (row_number() OVER w1 <= 1) |
| 11 | Storage: Memory Maximum Storage: 17kB |
| 12 | Buffers: shared hit=1664 |
| 13 | -> Incremental Sort (cost=4686.23..4774.37 rows=360 width=39) (actual time=47.640..52.093 rows=279.00 loops=1) |
| 14 | Sort Key: collection_counts.body_type, collection_counts.collections DESC, collection_counts.station_id |
| 15 | Presorted Key: collection_counts.body_type |
| 16 | Full-sort Groups: 6 Sort Method: quicksort Average Memory: 28kB Peak Memory: 28kB |
| 17 | Buffers: shared hit=1664 |
| 18 | -> Subquery Scan on collection_counts (cost=4685.83..4763.77 rows=360 width=39) (actual time=47.589..52.009 rows=279.00 loops=1) |
| 19 | Buffers: shared hit=1664 |
| 20 | -> Finalize GroupAggregate (cost=4685.83..4763.77 rows=360 width=39) (actual time=47.588..51.992 rows=279.00 loops=1) |
| 21 | Group Key: m.body_type, s.station_id |
| 22 | Buffers: shared hit=1664 |
| 23 | -> Gather Merge (cost=4685.83..4755.58 rows=612 width=39) (actual time=47.582..51.930 rows=558.00 loops=1) |
| 24 | Workers Planned: 1 |
| 25 | Workers Launched: 1 |
| 26 | Buffers: shared hit=1664 |
| 27 | -> Sort (cost=3685.82..3686.72 rows=360 width=39) (actual time=27.239..27.248 rows=279.00 loops=2) |
| 28 | Sort Key: m.body_type, s.station_id |
| 29 | Sort Method: quicksort Memory: 41kB |
| 30 | Buffers: shared hit=1664 |
| 31 | Worker 0: Sort Method: quicksort Memory: 41kB |
| 32 | -> Partial HashAggregate (cost=3666.93..3670.53 rows=360 width=39) (actual time=26.755..26.773 rows=279.00 loops=2) |
| 33 | Group Key: m.body_type, s.station_id |
| 34 | Batches: 1 Memory Usage: 72kB |

| | QUERY PLAN text |
|---|---|
| 35 | Buffers: shared hit=1656 |
| 36 | Worker 0: Batches: 1 Memory Usage: 72kB |
| 37 | -> Hash Join (cost=32.62..3404.07 rows=35048 width=31) (actual time=0.838..23.307 rows=29769.50 loops=2) |
| 38 | Hash Cond: ((v.station_id)::text = (s.station_id)::text) |
| 39 | Buffers: shared hit=1656 |
| 40 | -> Hash Join (cost=30.27..3303.22 rows=35048 width=14) (actual time=0.612..20.362 rows=29769.50 loops=2) |
| 41 | Hash Cond: ((v.model_code)::text = (m.model_code)::text) |
| 42 | Buffers: shared hit=1654 |
| 43 | -> Hash Join (cost=27.25..3203.87 rows=35048 width=15) (actual time=0.435..17.595 rows=29769.50 loops=2) |
| 44 | Hash Cond: ((e.vehicle_id)::text = (v.vehicle_id)::text) |
| 45 | Buffers: shared hit=1652 |
| 46 | -> Parallel Seq Scan on booking_events e (cost=0.00..3084.18 rows=35048 width=9) (actual time=0.143..14.286 rows=29769.50 loops=2) |
| 47 | Filter: ((event_type)::text = 'Collect'::text) |
| 48 | Rows Removed by Filter: 68453 |
| 49 | Buffers: shared hit=1638 |
| 50 | -> Hash (cost=16.00..16.00 rows=900 width=24) (actual time=0.266..0.266 rows=900.00 loops=2) |
| 51 | Buckets: 1024 Batches: 1 Memory Usage: 58kB |
| 52 | Buffers: shared hit=14 |
| 53 | -> Seq Scan on vehicles v (cost=0.00..16.00 rows=900 width=24) (actual time=0.125..0.194 rows=900.00 loops=2) |
| 54 | Buffers: shared hit=14 |
| 55 | -> Hash (cost=1.90..1.90 rows=90 width=15) (actual time=0.171..0.171 rows=90.00 loops=2) |
| 56 | Buckets: 1024 Batches: 1 Memory Usage: 13kB |
| 57 | Buffers: shared hit=2 |
| 58 | -> Seq Scan on models m (cost=0.00..1.90 rows=90 width=15) (actual time=0.158..0.162 rows=90.00 loops=2) |
| 59 | Buffers: shared hit=2 |
| 60 | -> Hash (cost=1.60..1.60 rows=60 width=24) (actual time=0.219..0.220 rows=60.00 loops=2) |
| 61 | Buckets: 1024 Batches: 1 Memory Usage: 12kB |
| 62 | Buffers: shared hit=2 |
| 63 | -> Seq Scan on stations s (cost=0.00..1.60 rows=60 width=24) (actual time=0.205..0.207 rows=60.00 loops=2) |
| 64 | Buffers: shared hit=2 |
| 65 | Planning: |
| 66 | Buffers: shared hit=203 dirtied=1 |
| 67 | Planning Time: 7.797 ms |
| 68 | Execution Time: 52.686 ms |

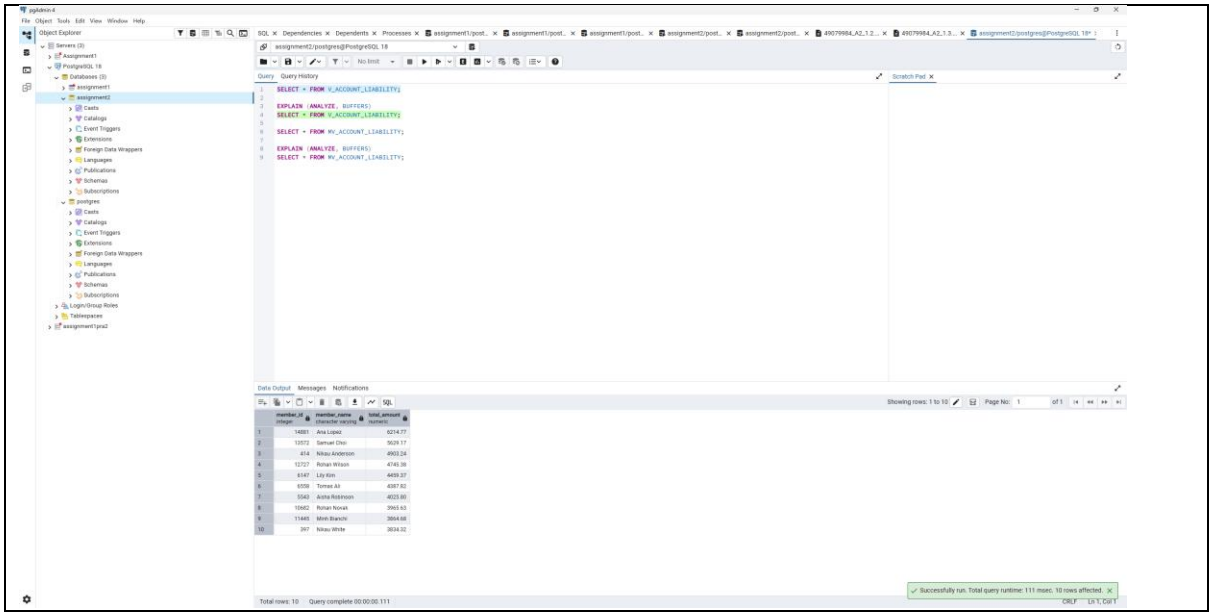
Comments:

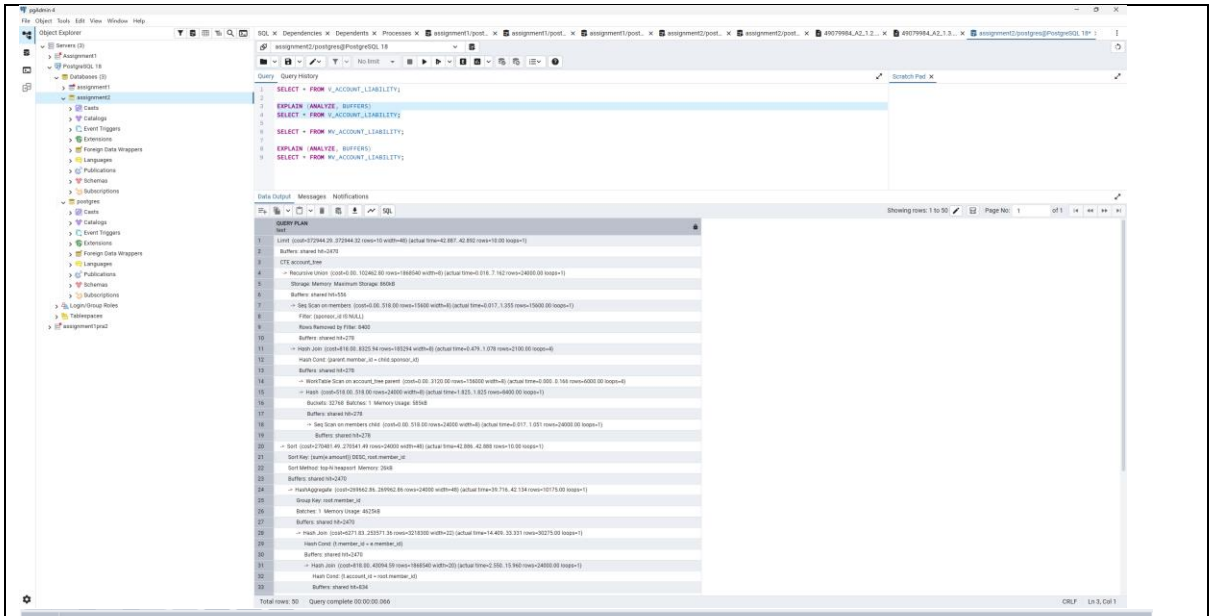
| Screenshot Q1.2 | Marks: /1.5 |
|---|---|



Comments:

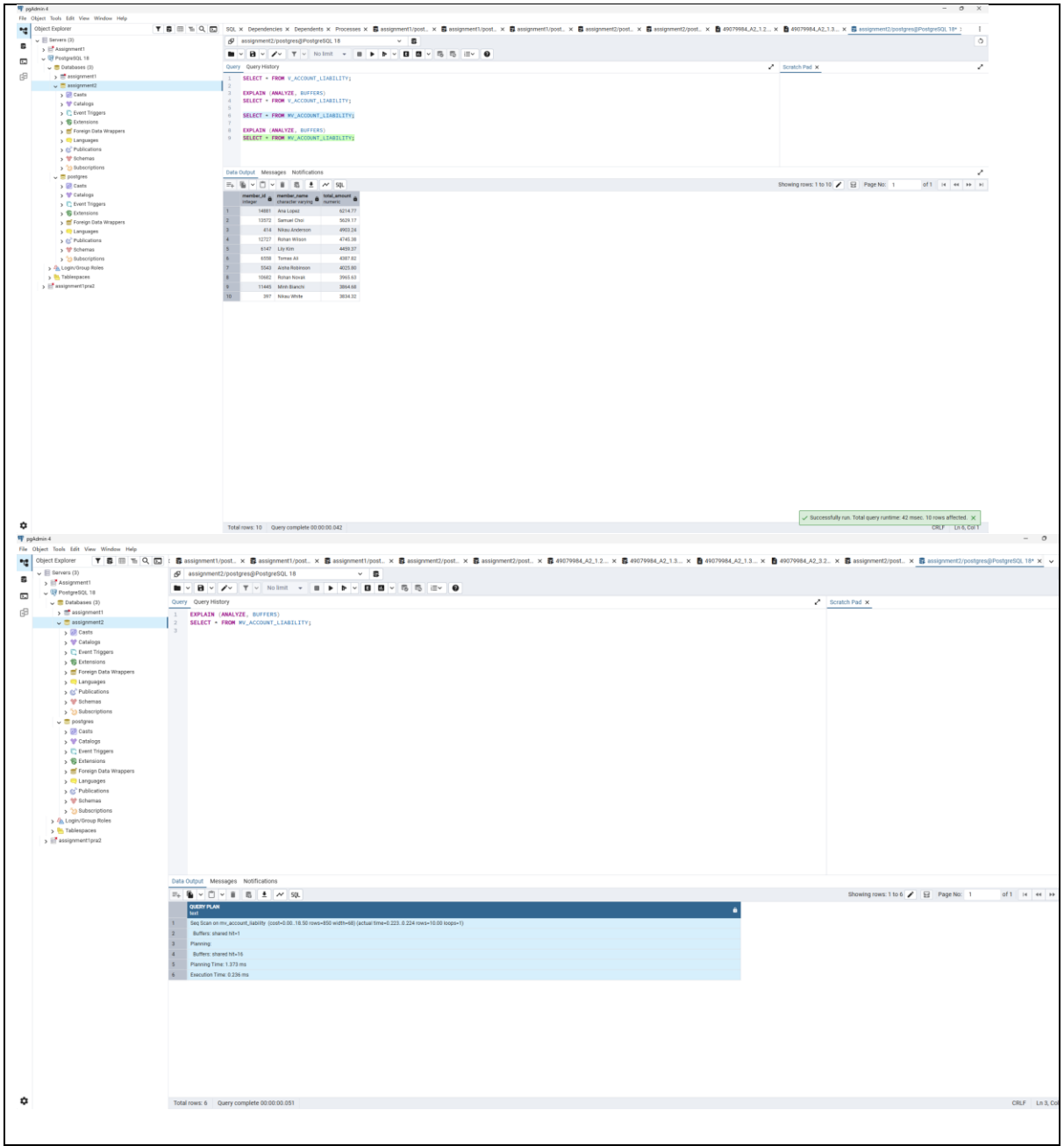
| Screenshot Q1.3 (2) | Marks: /0.5 |
|---|---|


SELECT * FROM V_ACCOUNT_LIABILITY;
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM V_ACCOUNT_LIABILITY;
SELECT * FROM MV_ACCOUNT_LIABILITY;
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM MV_ACCOUNT_LIABILITY;
Data Output
Messages
Notifications
member_id integer
member_name character varying
total_amount numeric
14881 Ana Lopez 6214.77
13572 Samuel Choi 5629.17
414 Nikau Anderson 4903.24
12727 Rohan Wilson 4745.38
8147 Lily Kim 4459.37
6558 Tomas Ali 4387.82
5543 Anita Robinson 4025.80
10662 Rohan Novak 3965.63
11445 Minh Bianchi 3864.68
397 Nikau White 3834.02
Successfully run. Total query runtime: 111 msec. 10 rows affected.
Total rows: 10
Query complete 00:00:00.111



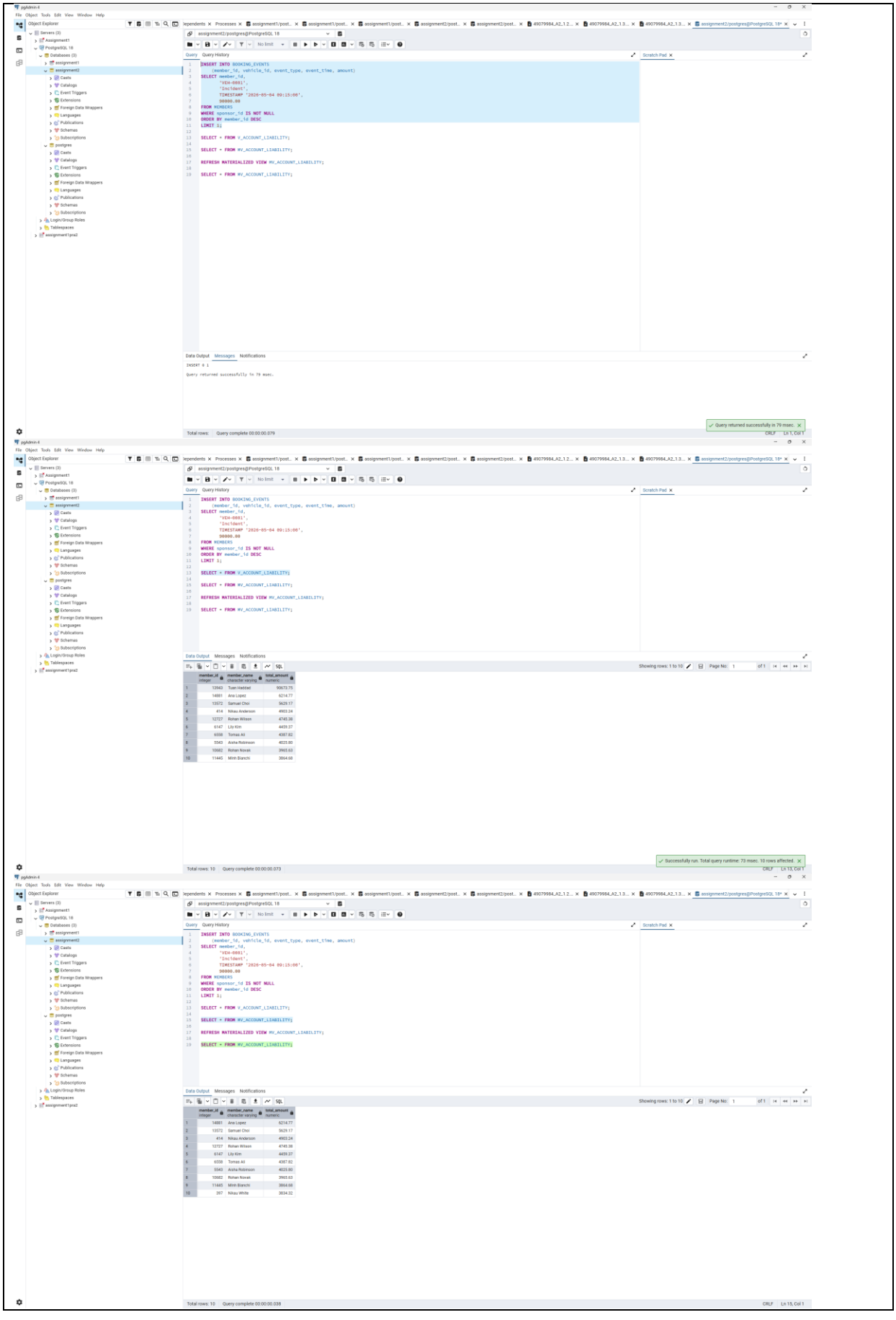
| | QUERY PLAN text |
|---|---|
| 1 | Limit (cost=370929.67..370929.69 rows=10 width=48) (actual time=44.446..44.453 rows=10.00 loops=1) |
| 2 | Buffers: shared hit=1769 |
| 3 | CTE account_tree |
| 4 | -> Recursive Union (cost=0.00..102462.80 rows=1868540 width=8) (actual time=0.024..9.410 rows=24000.00 loops=1) |
| 5 | Storage: Memory Maximum Storage: 860kB |
| 6 | Buffers: shared hit=556 |
| 7 | -> Seq Scan on members (cost=0.00..518.00 rows=15600 width=8) (actual time=0.021..1.411 rows=15600.00 loops=1) |
| 8 | Filter: (sponsor_id IS NULL) |
| 9 | Rows Removed by Filter: 8400 |
| 10 | Buffers: shared hit=278 |
| 11 | -> Hash Join (cost=818.00..8325.94 rows=185294 width=8) (actual time=0.645..1.589 rows=2100.00 loops=4) |
| 12 | Hash Cond: (parent.member_id = child.sponsor_id) |
| 13 | Buffers: shared hit=278 |
| 14 | -> WorkTable Scan on account_tree parent (cost=0.00..3120.00 rows=156000 width=8) (actual time=0.001..0.160 rows=6000.00 loops=4) |
| 15 | -> Hash (cost=518.00..518.00 rows=24000 width=8) (actual time=2.469..2.470 rows=8400.00 loops=1) |
| 16 | Buckets: 32768 Batches: 1 Memory Usage: 585kB |
| 17 | Buffers: shared hit=278 |
| 18 | -> Seq Scan on members child (cost=0.00..518.00 rows=24000 width=8) (actual time=0.027..1.397 rows=24000.00 loops=1) |
| 19 | Buffers: shared hit=278 |
| 20 | -> Sort (cost=268466.87..268526.87 rows=24000 width=48) (actual time=44.445..44.449 rows=10.00 loops=1) |
| 21 | Sort Key: (sum(e.amount)) DESC, root.member_id |
| 22 | Sort Method: top-N heapsort Memory: 26kB |
| 23 | Buffers: shared hit=1769 |
| 24 | -> HashAggregate (cost=267648.24..267948.24 rows=24000 width=48) (actual time=40.579..43.523 rows=10175.00 loops=1) |
| 25 | Group Key: root.member_id |
| 26 | Batches: 1 Memory Usage: 4625kB |
| 27 | Buffers: shared hit=1769 |
| 28 | -> Hash Join (cost=4198.46..251537.16 rows=3222216 width=22) (actual time=8.591..32.947 rows=30276.00 loops=1) |
| 29 | Hash Cond: (t.member_id = e.member_id) |
| 30 | Buffers: shared hit=1769 |
| 31 | -> Hash Join (cost=818.00..43094.59 rows=1868540 width=20) (actual time=2.874..19.526 rows=24000.00 loops=1) |
| 32 | Hash Cond: (t.account_id = root.member_id) |
| 33 | Buffers: shared hit=834 |
| 34 | -> CTE Scan on account_tree t (cost=0.00..37370.80 rows=1868540 width=8) (actual time=0.026..12.015 rows=24000.00 loops=1) |

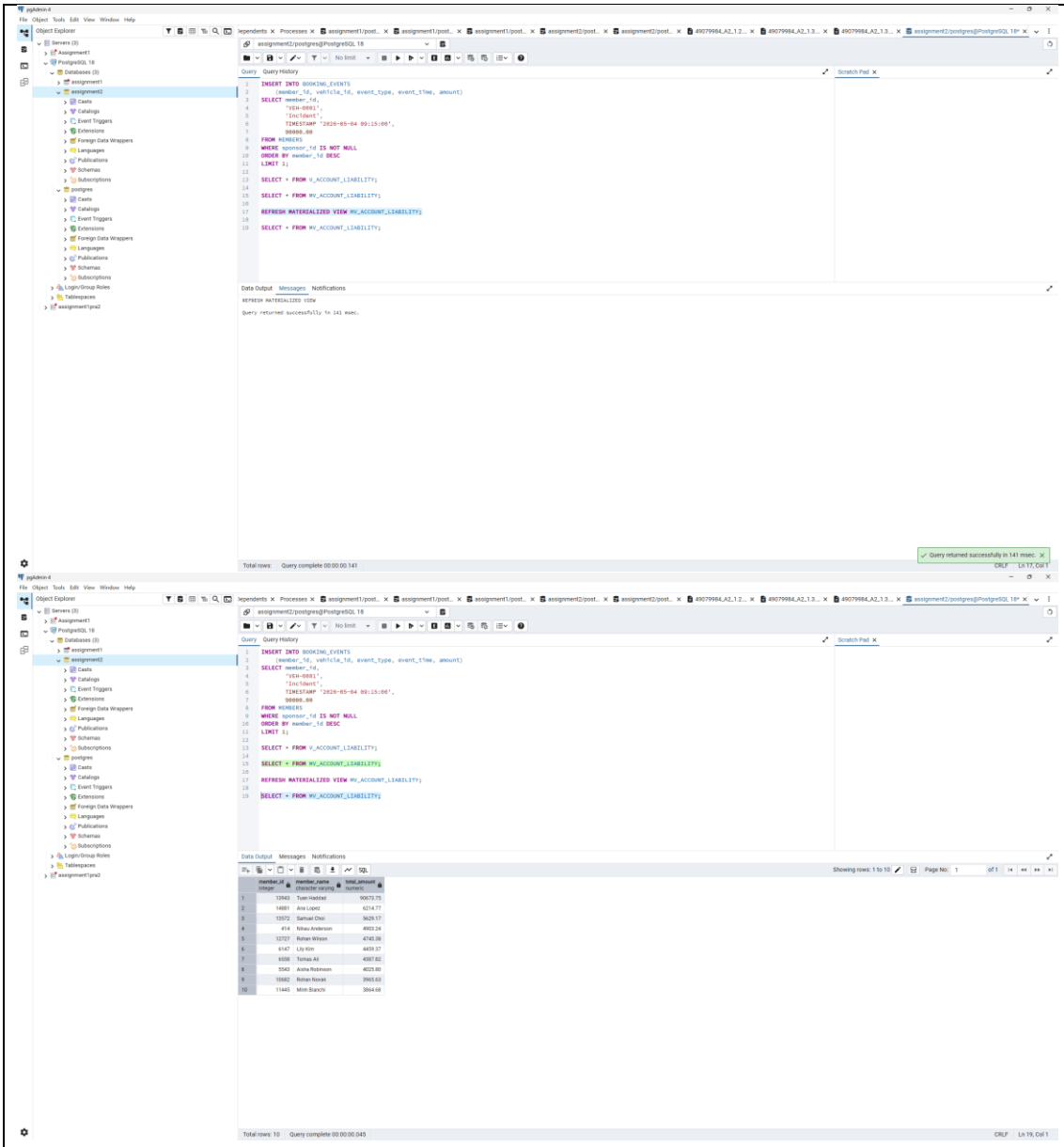
| | QUERY PLAN<br>text |
|---|---|
| 21 | Sort Key: (sum(e.amount)) DESC, root.member_id |
| 22 | Sort Method: top-N heapsort Memory: 26kB |
| 23 | Buffers: shared hit=1769 |
| 24 | -> HashAggregate (cost=267648.24..267948.24 rows=24000 width=48) (actual time=40.579..43.523 rows=10175.00 loops=1) |
| 25 | Group Key: root.member_id |
| 26 | Batches: 1 Memory Usage: 4625kB |
| 27 | Buffers: shared hit=1769 |
| 28 | -> Hash Join (cost=4198.46..251537.16 rows=3222216 width=22) (actual time=8.591..32.947 rows=30276.00 loops=1) |
| 29 | Hash Cond: (t.member_id = e.member_id) |
| 30 | Buffers: shared hit=1769 |
| 31 | -> Hash Join (cost=818.00..43094.59 rows=1868540 width=20) (actual time=2.874..19.526 rows=24000.00 loops=1) |
| 32 | Hash Cond: (t.account_id = root.member_id) |
| 33 | Buffers: shared hit=834 |
| 34 | -> CTE Scan on account_tree t (cost=0.00..37370.80 rows=1868540 width=8) (actual time=0.026..12.015 rows=24000.00 loops=1) |
| 35 | Storage: Memory Maximum Storage: 1007kB |
| 36 | Buffers: shared hit=556 |
| 37 | -> Hash (cost=518.00..518.00 rows=24000 width=16) (actual time=2.790..2.791 rows=24000.00 loops=1) |
| 38 | Buckets: 32768 Batches: 1 Memory Usage: 1400kB |
| 39 | Buffers: shared hit=278 |
| 40 | -> Seq Scan on members root (cost=0.00..518.00 rows=24000 width=16) (actual time=0.004..1.138 rows=24000.00 loops=1) |
| 41 | Buffers: shared hit=278 |
| 42 | -> Hash (cost=2999.87..2999.87 rows=30447 width=10) (actual time=5.666..5.667 rows=30276.00 loops=1) |
| 43 | Buckets: 32768 Batches: 1 Memory Usage: 1557kB |
| 44 | Buffers: shared hit=935 |
| 45 | -> Bitmap Heap Scan on booking_events e (cost=829.05..2999.87 rows=30447 width=10) (actual time=0.616..3.690 rows=30276.00 loops=1) |
| 46 | Recheck Cond: (((event_type)::text = ANY ('{Return,Incident}'::text[])) AND (event_time >= '2026-01-01 00:00:00'::timestamp without time zone) AND (event_time < '2026-0... |
| 47 | Heap Blocks: exact=811 |
| 48 | Buffers: shared hit=935 |
| 49 | -> Bitmap Index Scan on idx_event_type_time (cost=0.00..821.44 rows=30447 width=0) (actual time=0.555..0.556 rows=30276.00 loops=1) |
| 50 | Index Cond: (((event_type)::text = ANY ('{Return,Incident}'::text[])) AND (event_time >= '2026-01-01 00:00:00'::timestamp without time zone) AND (event_time < '20... |
| 51 | Index Searches: 2 |
| 52 | Buffers: shared hit=124 |
| 53 | Planning Time: 0.384 ms |
| 54 | Execution Time: 45.543 ms |



Comments:

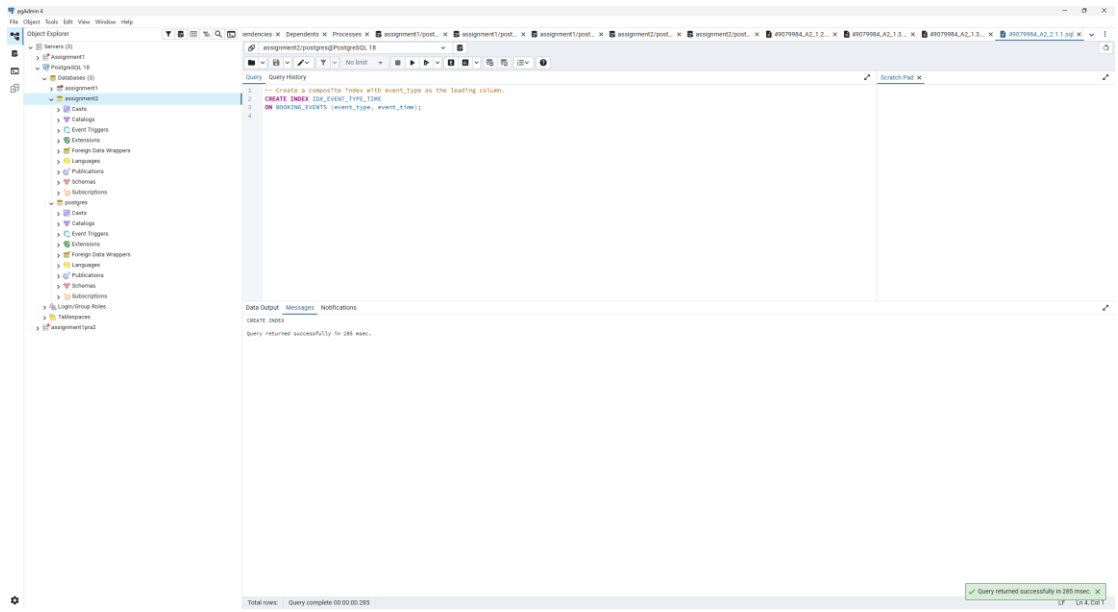
| Screenshot Q1.3 (3) | Marks: /0.5 |
|---|---|


pgAdmin 4
Object Explorer
assignment2/postgres@PostgreSQL 18
Query
Query History
Scratch Pad
INSERT INTO BOOKING_EVENTS
(member_id, vehicle_id, event_type, event_time, amount)
SELECT member_id,
'YEH-0881',
'Inclident',
TIMESTAMP '2026-05-04 09:15:00',
90000.00
FROM MEMBERS
WHERE sponsor_id IS NOT NULL
ORDER BY member_id DESC
LIMIT 1;
SELECT * FROM V_ACCOUNT_LIABILITY;
SELECT * FROM MV_ACCOUNT_LIABILITY;
REFRESH MATERIALIZED VIEW MV_ACCOUNT_LIABILITY;
SELECT * FROM MV_ACCOUNT_LIABILITY;
Data Output
Messages
Notifications
INSERT 0 1
Query returned successfully in 79 msec.
Total rows:
Query complete 00:00:00.079
Query returned successfully in 79 msec.
CRLF
Ln 1, Col 1
member_id integer
member_name character varying
total_amount numeric
1 13943 Tuan Haddad 90673.75
2 14881 Ana Lopez 6214.77
3 13572 Samuel Choi 5629.17
4 414 Nikau Anderson 4903.24
5 12727 Rohan Wilson 4745.38
6 6147 Lily Kim 4459.37
7 6058 Tomas Ali 4387.82
8 5543 Aisha Robinson 4025.80
9 10682 Rohan Novak 3965.63
10 11445 Minh Bianchi 3864.68
Showing rows: 1 to 10
Page No: 1
of 1
Successfully run. Total query runtime: 73 msec. 10 rows affected.
Total rows: 10
Query complete 00:00:00.073
Ln 13, Col 1
member_id integer
member_name character varying
total_amount numeric
1 14881 Ana Lopez 6214.77
2 13572 Samuel Choi 5629.17
3 414 Nikau Anderson 4903.24
4 12727 Rohan Wilson 4745.38
5 6147 Lily Kim 4459.37
6 6058 Tomas Ali 4387.82
7 5543 Aisha Robinson 4025.80
8 10682 Rohan Novak 3965.63
9 11445 Minh Bianchi 3864.68
10 367 Nikau White 3834.32
Total rows: 10
Query complete 00:00:00.038
Ln 15, Col 1



Comments:

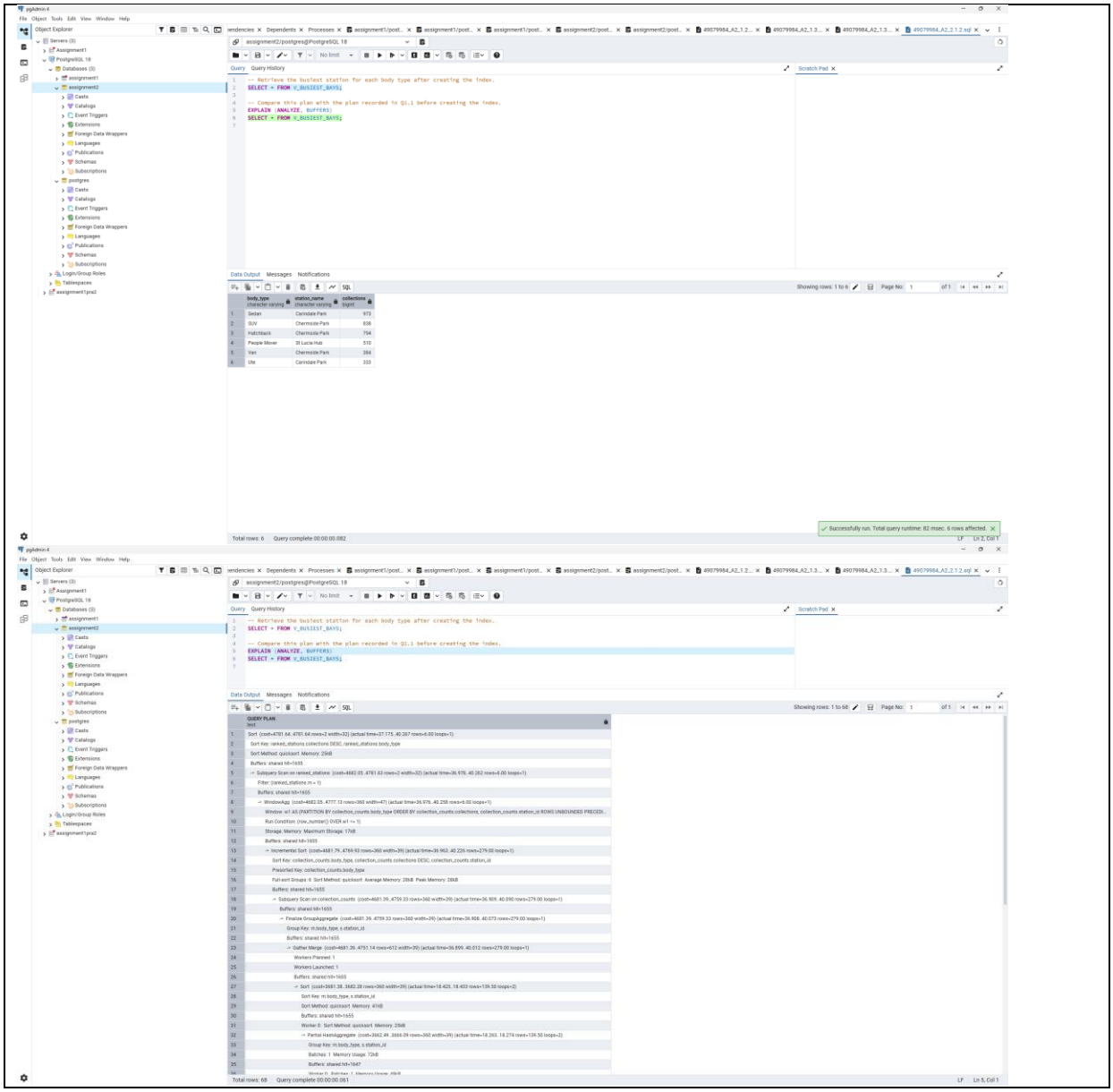
| Screenshot Q2.1 (1) | Marks: /0.5 |
|---|---|




This B-tree index has event_type as its first key and event_time as its second key, in the required order. It orders timestamps within each event type

Comments:

| Screenshot Q2.1 (2) | Marks: /1.5 |
|---|---|

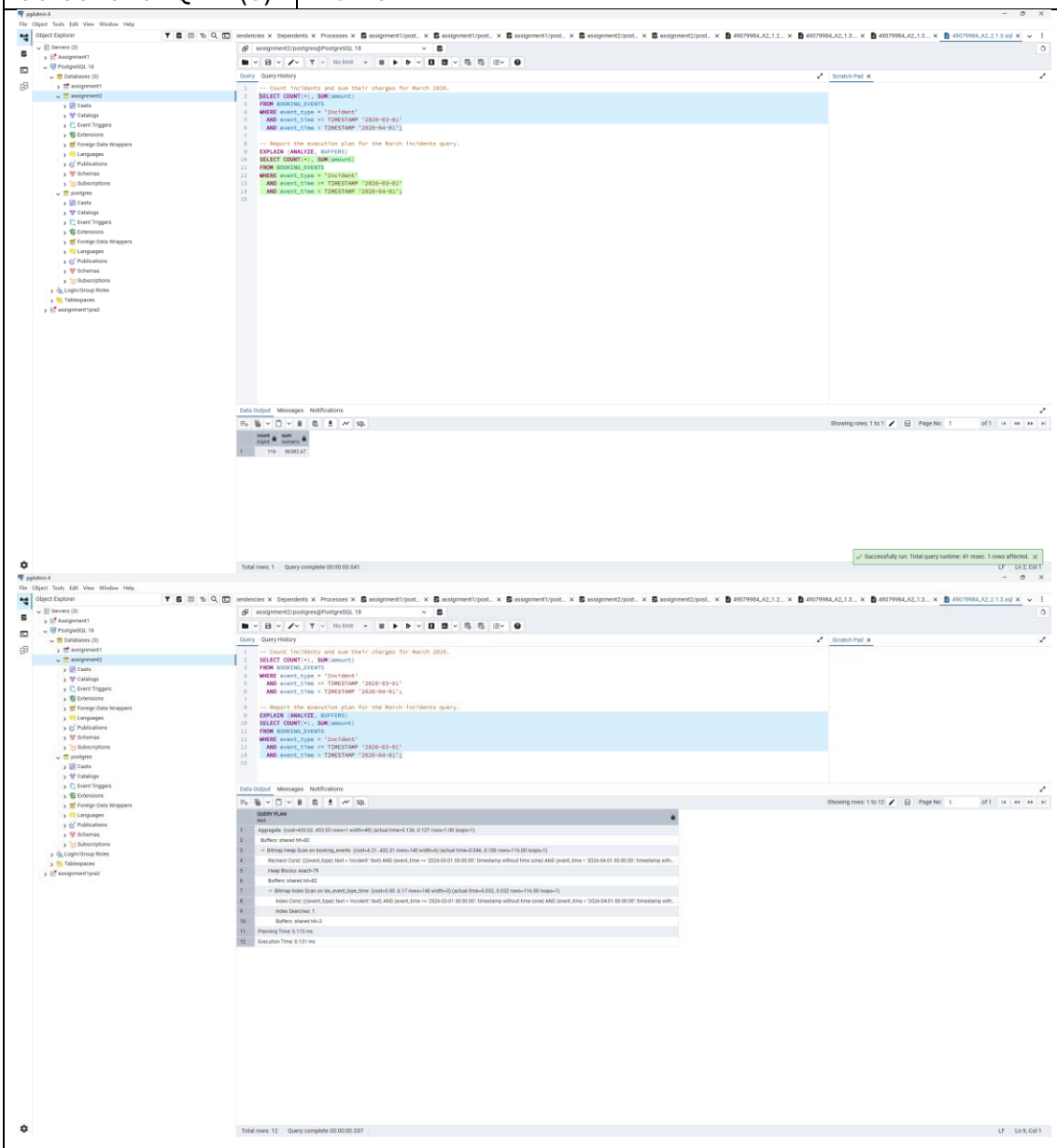

-- Retrieve the busiest station for each body type after creating the index.
SELECT * FROM V_BUSIEST_DAYS;
-- Compare this plan with the plan recorded in Q1.1 before creating the index.
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM V_BUSIEST_DAYS;
Data Output
Messages
Notifications
Query Plan
Sort (cost=4781.64..4781.64 rows=2 width=32) (actual time=37.175..40.267 rows=6.00 loops=1)
Sort Key: ranked_stations.collections DESC, ranked_stations.body_type
Sort Method: quicksort Memory: 25kB
Buffers: shared hit=1655
Subquery Scan on ranked_stations (cost=4682.05..4781.63 rows=2 width=32) (actual time=36.978..40.262 rows=6.00 loops=1)
Filter: (ranked_stations.rn = 1)
Buffers: shared hit=1655
WindowAgg (cost=4682.05..4777.13 rows=360 width=47) (actual time=36.976..40.258 rows=6.00 loops=1)
Window: w1 AS (PARTITION BY collection_counts.body_type ORDER BY collection_counts.collections, collection_counts.station_id ROWS UNBOUNDED PRECEDI...
Run Condition: (row_number() OVER w1 <= 1)
Storage: Memory Maximum Storage: 17kB
Buffers: shared hit=1655
Incremental Sort (cost=4681.79..4769.93 rows=360 width=39) (actual time=36.962..40.226 rows=279.00 loops=1)
Sort Key: collection_counts.body_type, collection_counts.collections DESC, collection_counts.station_id
Presorted Key: collection_counts.body_type
Full-sort Groups: 6 Sort Method: quicksort Average Memory: 28kB Peak Memory: 28kB
Buffers: shared hit=1655
Subquery Scan on collection_counts (cost=4681.39..4759.33 rows=360 width=39) (actual time=36.909..40.090 rows=279.00 loops=1)
Buffers: shared hit=1655
Finalize GroupAggregate (cost=4681.39..4759.33 rows=360 width=39) (actual time=36.908..40.073 rows=279.00 loops=1)
Group Key: m.body_type, s.station_id
Buffers: shared hit=1655
Gather Merge (cost=4681.39..4751.14 rows=612 width=39) (actual time=36.899..40.012 rows=279.00 loops=1)
Workers Planned: 1
Workers Launched: 1
Buffers: shared hit=1655
Sort (cost=3681.38..3682.28 rows=360 width=39) (actual time=18.425..18.433 rows=139.50 loops=2)
Sort Key: m.body_type, s.station_id
Sort Method: quicksort Memory: 41kB
Buffers: shared hit=1655
Worker 0: Sort Method: quicksort Memory: 29kB
Partial HashAggregate (cost=3662.49..3666.09 rows=360 width=39) (actual time=18.263..18.274 rows=139.50 loops=2)
Group Key: m.body_type, s.station_id
Batches: 1 Memory Usage: 72kB
Buffers: shared hit=1647
Total rows: 68 Query complete 00:00:00.061
Ln 5, Col 1

| | QUERY PLAN<br>text |
|---|---|
| 1 | Sort  (cost=4786.08..4786.08 rows=2 width=32) (actual time=37.491..40.432 rows=6.00 loops=1) |
| 2 | Sort Key: ranked_stations.collections DESC, ranked_stations.body_type |
| 3 | Sort Method: quicksort  Memory: 25kB |
| 4 | Buffers: shared hit=1664 |
| 5 | -> Subquery Scan on ranked_stations  (cost=4686.49..4786.07 rows=2 width=32) (actual time=37.303..40.427 rows=6.00 loops=1) |
| 6 | Filter: (ranked_stations.rn = 1) |
| 7 | Buffers: shared hit=1664 |
| 8 | -> WindowAgg  (cost=4686.49..4781.57 rows=360 width=47) (actual time=37.302..40.425 rows=6.00 loops=1) |
| 9 | Window: w1 AS (PARTITION BY collection_counts.body_type ORDER BY collection_counts.collections, collection_counts.station_id ROWS UNBOUNDED PRECEDING) |
| 10 | Run Condition: (row_number() OVER w1 <= 1) |
| 11 | Storage: Memory  Maximum Storage: 17kB |
| 12 | Buffers: shared hit=1664 |
| 13 | -> Incremental Sort  (cost=4686.23..4774.37 rows=360 width=39) (actual time=37.292..40.403 rows=279.00 loops=1) |
| 14 | Sort Key: collection_counts.body_type, collection_counts.collections DESC, collection_counts.station_id |
| 15 | Presorted Key: collection_counts.body_type |
| 16 | Full-sort Groups: 6  Sort Method: quicksort  Average Memory: 28kB  Peak Memory: 28kB |
| 17 | Buffers: shared hit=1664 |
| 18 | -> Subquery Scan on collection_counts  (cost=4685.83..4763.77 rows=360 width=39) (actual time=37.245..40.319 rows=279.00 loops=1) |
| 19 | Buffers: shared hit=1664 |
| 20 | -> Finalize GroupAggregate  (cost=4685.83..4763.77 rows=360 width=39) (actual time=37.244..40.299 rows=279.00 loops=1) |
| 21 | Group Key: m.body_type, s.station_id |
| 22 | Buffers: shared hit=1664 |
| 23 | -> Gather Merge  (cost=4685.83..4755.58 rows=612 width=39) (actual time=37.234..40.251 rows=330.00 loops=1) |
| 24 | Workers Planned: 1 |
| 25 | Workers Launched: 1 |
| 26 | Buffers: shared hit=1664 |
| 27 | -> Sort  (cost=3685.82..3686.72 rows=360 width=39) (actual time=19.089..19.096 rows=165.00 loops=2) |
| 28 | Sort Key: m.body_type, s.station_id |
| 29 | Sort Method: quicksort  Memory: 41kB |
| 30 | Buffers: shared hit=1664 |
| 31 | Worker 0:  Sort Method: quicksort  Memory: 27kB |
| 32 | -> Partial HashAggregate  (cost=3666.93..3670.53 rows=360 width=39) (actual time=18.906..18.918 rows=165.00 loops=2) |
| 33 | Group Key: m.body_type, s.station_id |
| 34 | Batches: 1  Memory Usage: 72kB |

| | QUERY PLAN<br>text |
|---|---|
| 35 | Buffers: shared hit=1656 |
| 36 | Worker 0:  Batches: 1  Memory Usage: 48kB |
| 37 | -> Hash Join  (cost=32.62..3404.07 rows=35048 width=31) (actual time=0.698..15.107 rows=29769.50 loops=2) |
| 38 | Hash Cond: ((v.station_id)::text = (s.station_id)::text) |
| 39 | Buffers: shared hit=1656 |
| 40 | -> Hash Join  (cost=30.27..3303.22 rows=35048 width=14) (actual time=0.464..11.802 rows=29769.50 loops=2) |
| 41 | Hash Cond: ((v.model_code)::text = (m.model_code)::text) |
| 42 | Buffers: shared hit=1654 |
| 43 | -> Hash Join  (cost=27.25..3203.87 rows=35048 width=15) (actual time=0.260..8.568 rows=29769.50 loops=2) |
| 44 | Hash Cond: ((e.vehicle_id)::text = (v.vehicle_id)::text) |
| 45 | Buffers: shared hit=1652 |
| 46 | -> Parallel Seq Scan on booking_events e  (cost=0.00..3084.18 rows=35048 width=9) (actual time=0.006..4.972 rows=29769.50 loops=2) |
| 47 | Filter: ((event_type)::text = 'Collect'::text) |
| 48 | Rows Removed by Filter: 68453 |
| 49 | Buffers: shared hit=1638 |
| 50 | -> Hash  (cost=16.00..16.00 rows=900 width=24) (actual time=0.249..0.249 rows=900.00 loops=2) |
| 51 | Buckets: 1024  Batches: 1  Memory Usage: 58kB |
| 52 | Buffers: shared hit=14 |
| 53 | -> Seq Scan on vehicles v  (cost=0.00..16.00 rows=900 width=24) (actual time=0.140..0.188 rows=900.00 loops=2) |
| 54 | Buffers: shared hit=14 |
| 55 | -> Hash  (cost=1.90..1.90 rows=90 width=15) (actual time=0.198..0.198 rows=90.00 loops=2) |
| 56 | Buckets: 1024  Batches: 1  Memory Usage: 13kB |
| 57 | Buffers: shared hit=2 |
| 58 | -> Seq Scan on models m  (cost=0.00..1.90 rows=90 width=15) (actual time=0.186..0.190 rows=90.00 loops=2) |
| 59 | Buffers: shared hit=2 |
| 60 | -> Hash  (cost=1.60..1.60 rows=60 width=24) (actual time=0.228..0.229 rows=60.00 loops=2) |
| 61 | Buckets: 1024  Batches: 1  Memory Usage: 12kB |
| 62 | Buffers: shared hit=2 |
| 63 | -> Seq Scan on stations s  (cost=0.00..1.60 rows=60 width=24) (actual time=0.215..0.218 rows=60.00 loops=2) |
| 64 | Buffers: shared hit=2 |
| 65 | Planning: |
| 66 | Buffers: shared hit=54 |
| 67 | Planning Time: 1.633 ms |
| 68 | Execution Time: 40.538 ms |

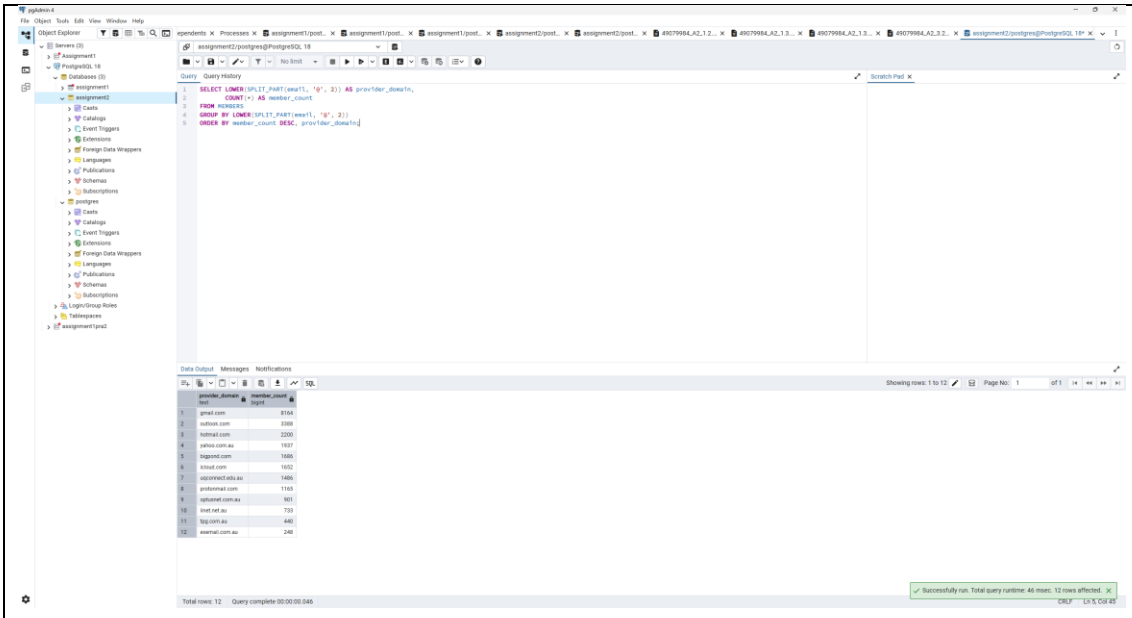
Comments:

| Screenshot Q2.1 (3) | Marks: /1 |
|---|---|



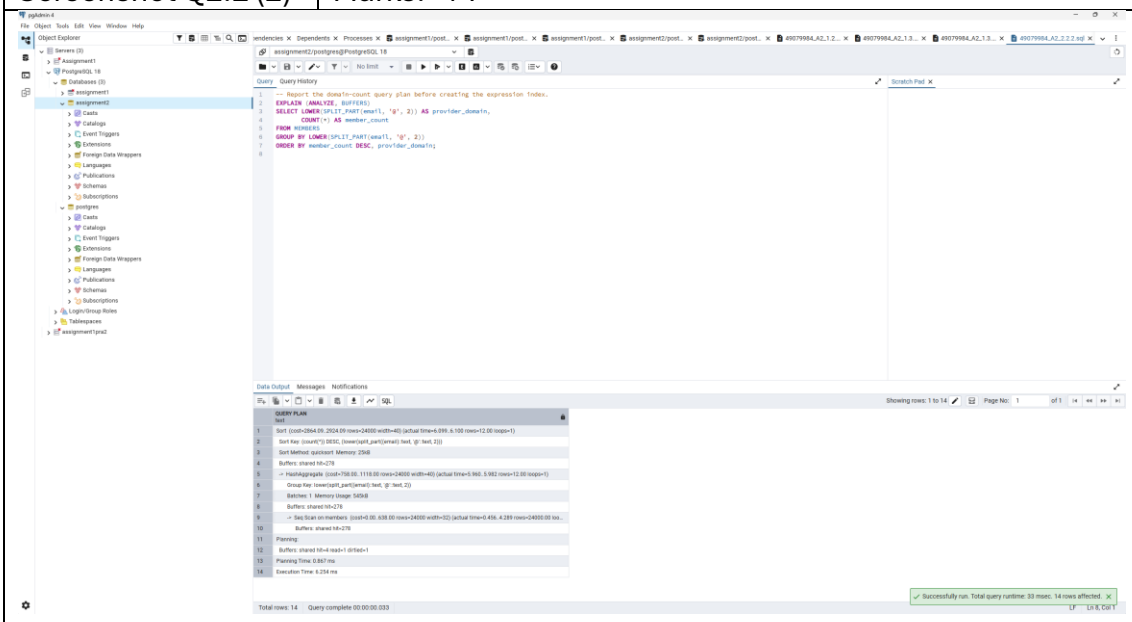
Comments:

| Screenshot Q2.2(1) | Marks: /1 |
|---|---|



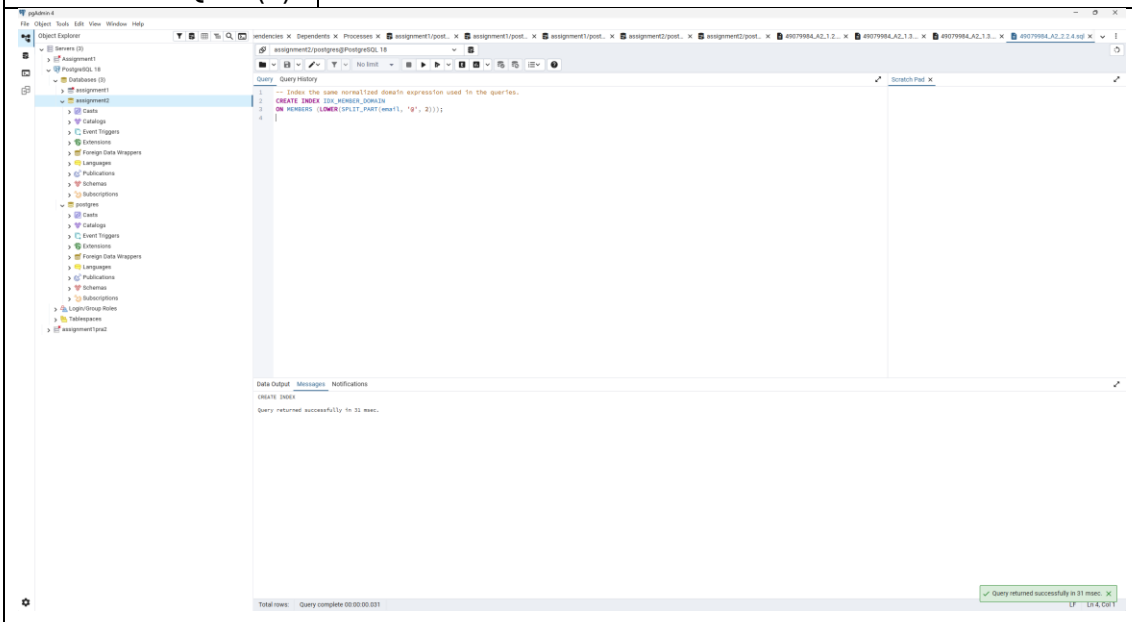
Comments:

| Screenshot Q2.2 (2) | Marks: /1 |
|---|---|



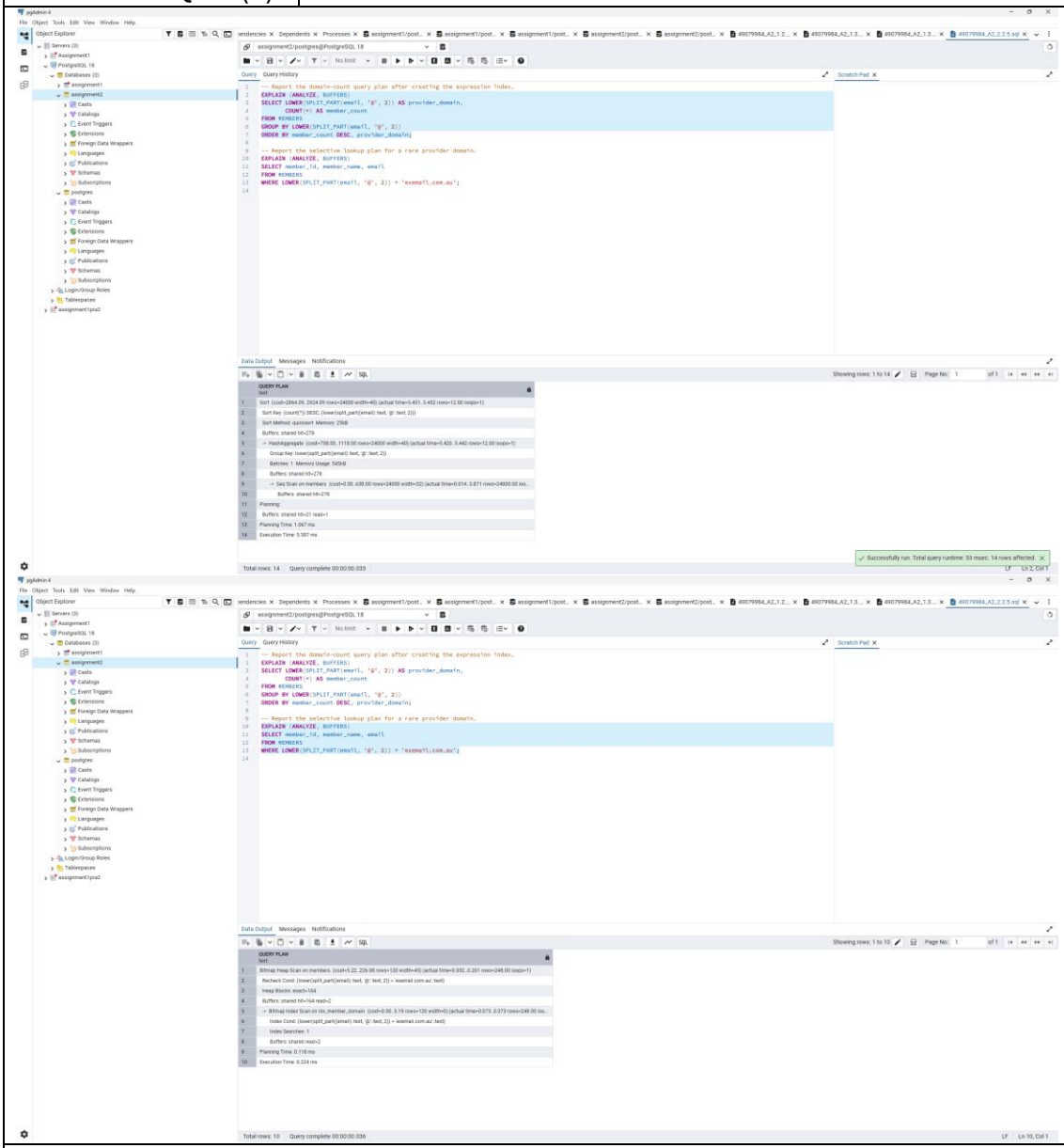
Comments:

| Screenshot Q2.2 (3) | Marks: /1 |
|---|---|
| The UNIQUE index on email orders complete email addresses, so members with the same provider domain are not necessarily adjacent. It does not index the normalized domain expression, so it cannot directly locate a domain or avoid processing every address for the domain-count query. | |
| Comments: | |

| Screenshot Q2.2 (4) | Marks: /0.5 |
|---|---|
|  | |

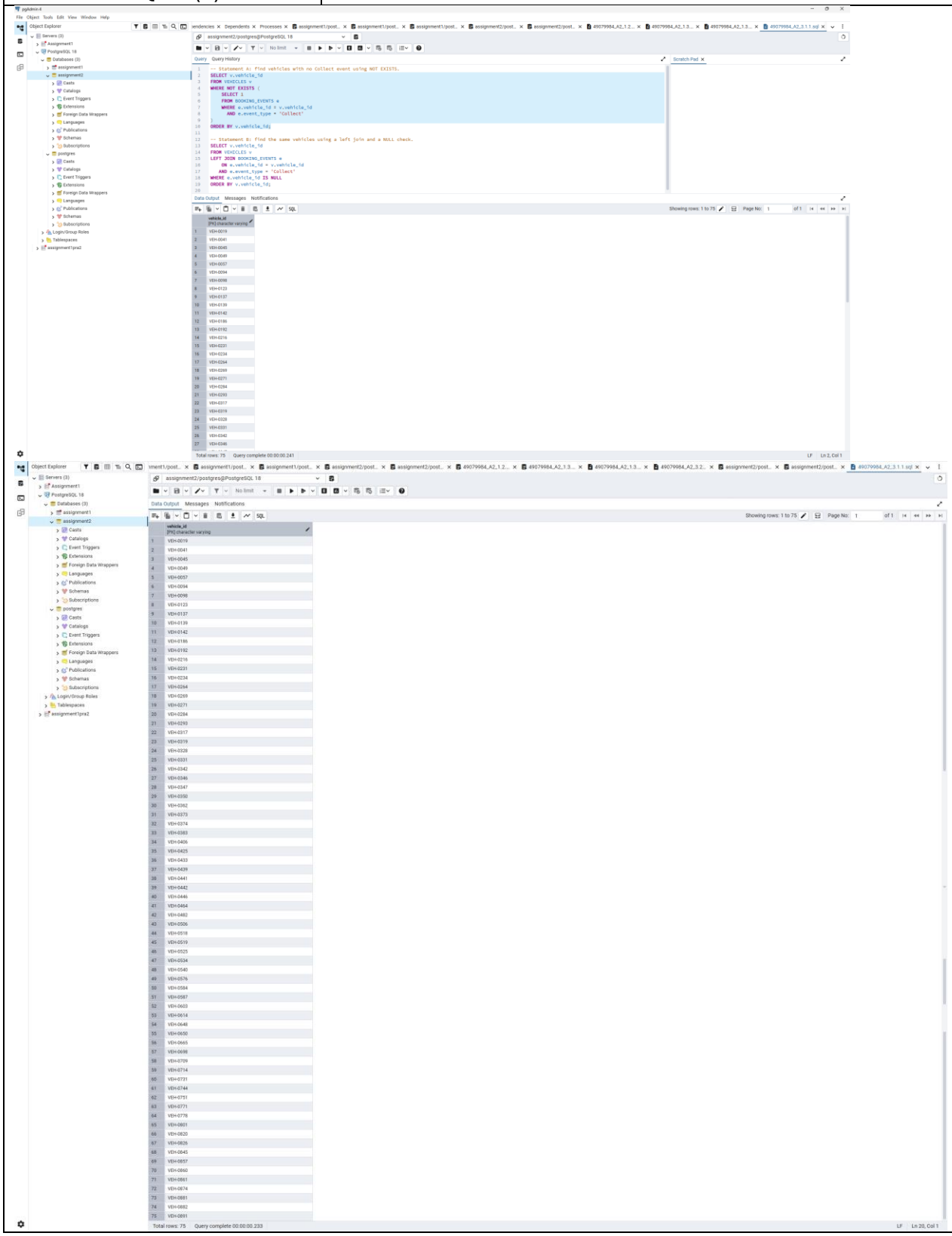
Comments:

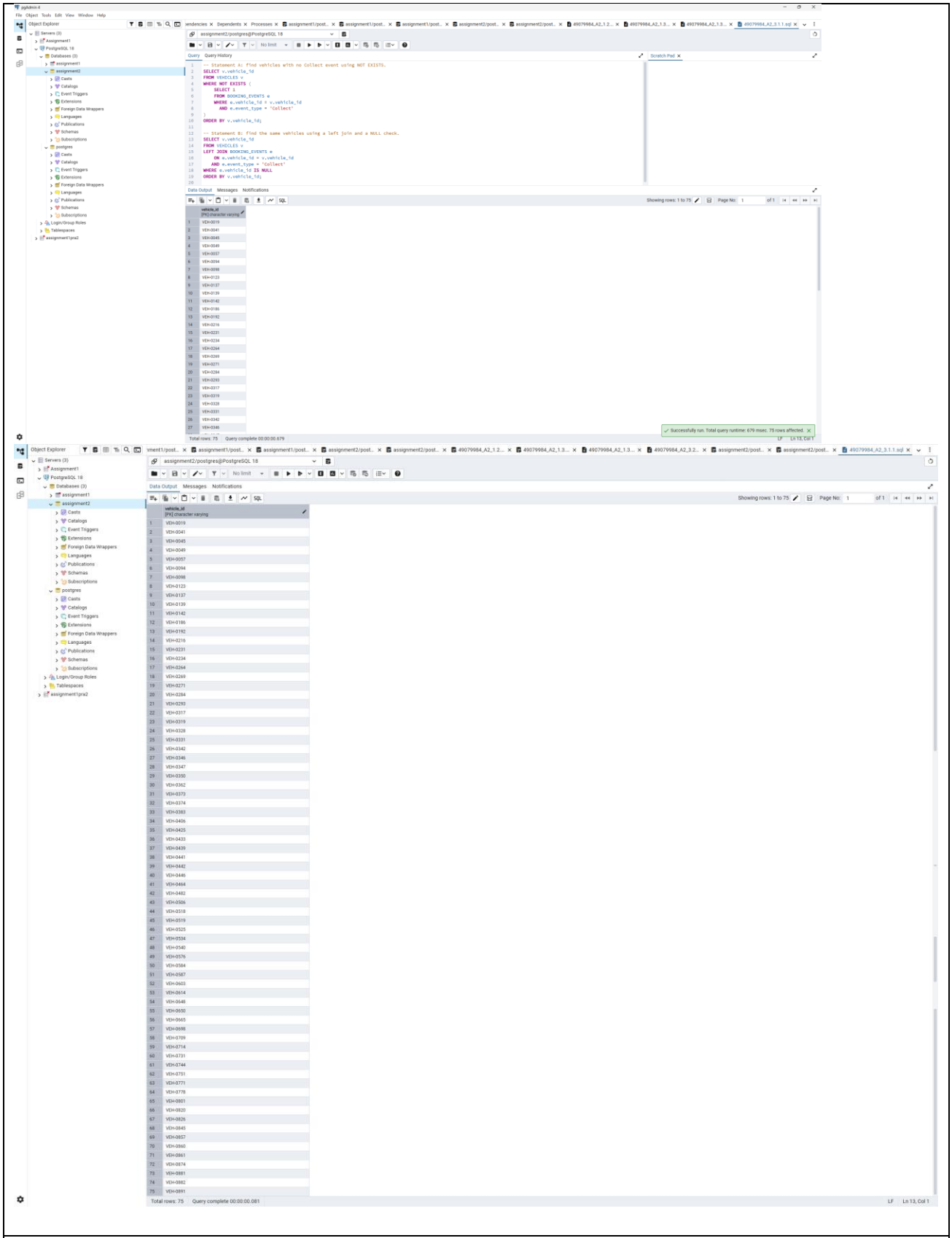
| Screenshot Q2.2 (5) | Marks: /1.5 |
|---|---|




Comments:

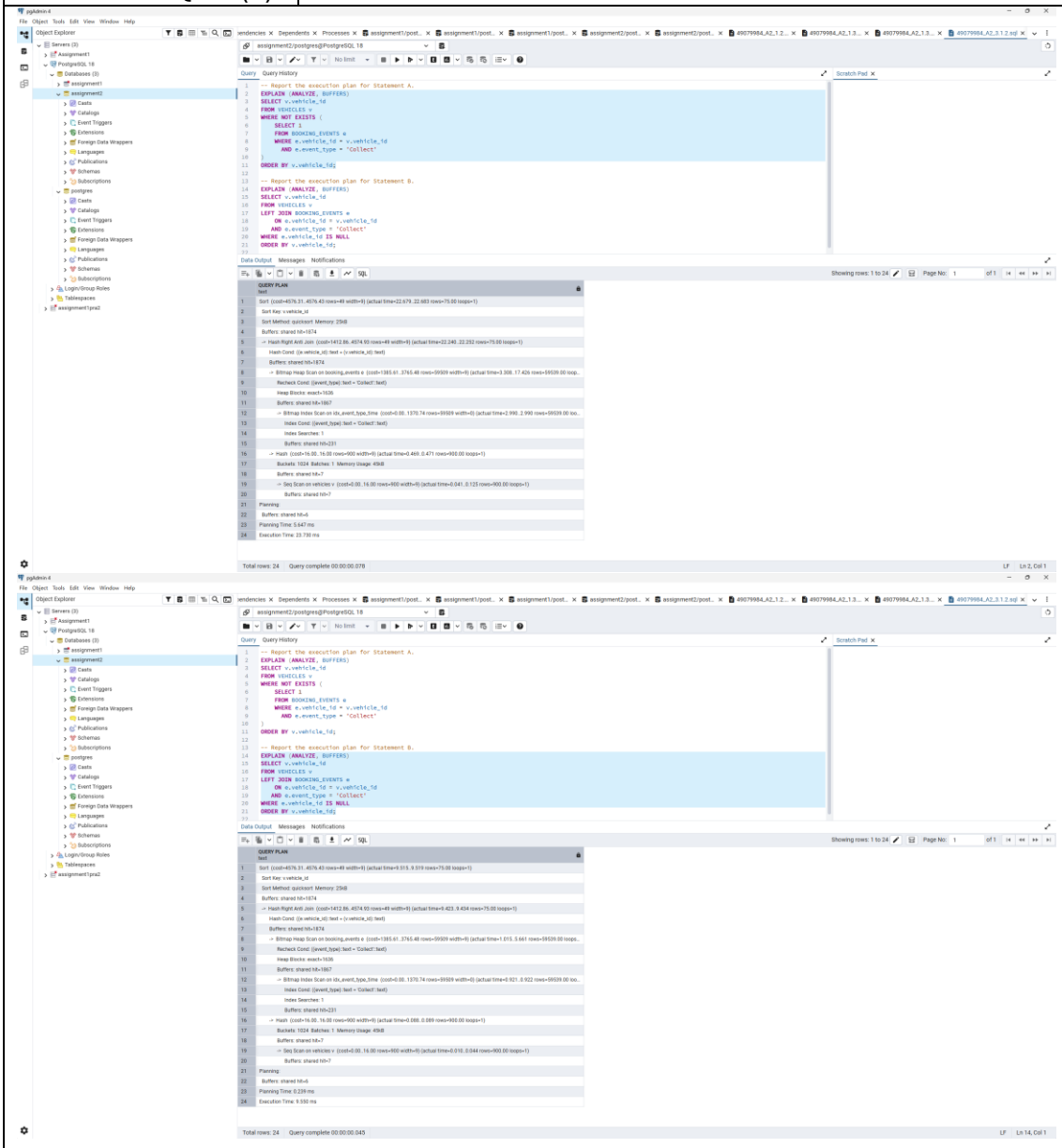
| Screenshot Q3.1 (1) | Marks: /1.5 |
|---|---|




-- Statement A: find vehicles with no Collect event using NOT EXISTS.
SELECT v.vehicle_id
FROM VEHICLES v
WHERE NOT EXISTS (
SELECT 1
FROM BOOKING_EVENTS e
WHERE e.vehicle_id = v.vehicle_id
AND e.event_type = 'Collect'
)
ORDER BY v.vehicle_id;
-- Statement B: find the same vehicles using a left join and a NULL check.
SELECT v.vehicle_id
FROM VEHICLES v
LEFT JOIN BOOKING_EVENTS e
ON e.vehicle_id = v.vehicle_id
AND e.event_type = 'Collect'
WHERE e.vehicle_id IS NULL
ORDER BY v.vehicle_id;
vehicle_id
[PK] character varying
VEH-0019
VEH-0041
VEH-0045
VEH-0049
VEH-0057
VEH-0094
VEH-0098
VEH-0123
VEH-0137
VEH-0139
VEH-0142
VEH-0186
VEH-0192
VEH-0216
VEH-0231
VEH-0234
VEH-0264
VEH-0269
VEH-0271
VEH-0284
VEH-0293
VEH-0317
VEH-0319
VEH-0328
VEH-0331
VEH-0342
VEH-0346
VEH-0347
VEH-0350
VEH-0362
VEH-0373
VEH-0374
VEH-0383
VEH-0406
VEH-0425
VEH-0433
VEH-0439
VEH-0441
VEH-0442
VEH-0446
VEH-0464
VEH-0482
VEH-0506
VEH-0518
VEH-0519
VEH-0525
VEH-0534
VEH-0540
VEH-0576
VEH-0584
VEH-0587
VEH-0603
VEH-0614
VEH-0648
VEH-0650
VEH-0665
VEH-0698
VEH-0709
VEH-0714
VEH-0731
VEH-0744
VEH-0751
VEH-0771
VEH-0778
VEH-0801
VEH-0820
VEH-0826
VEH-0845
VEH-0857
VEH-0860
VEH-0861
VEH-0874
VEH-0881
VEH-0882
VEH-0891
Total rows: 75
Query complete 00:00:00.081

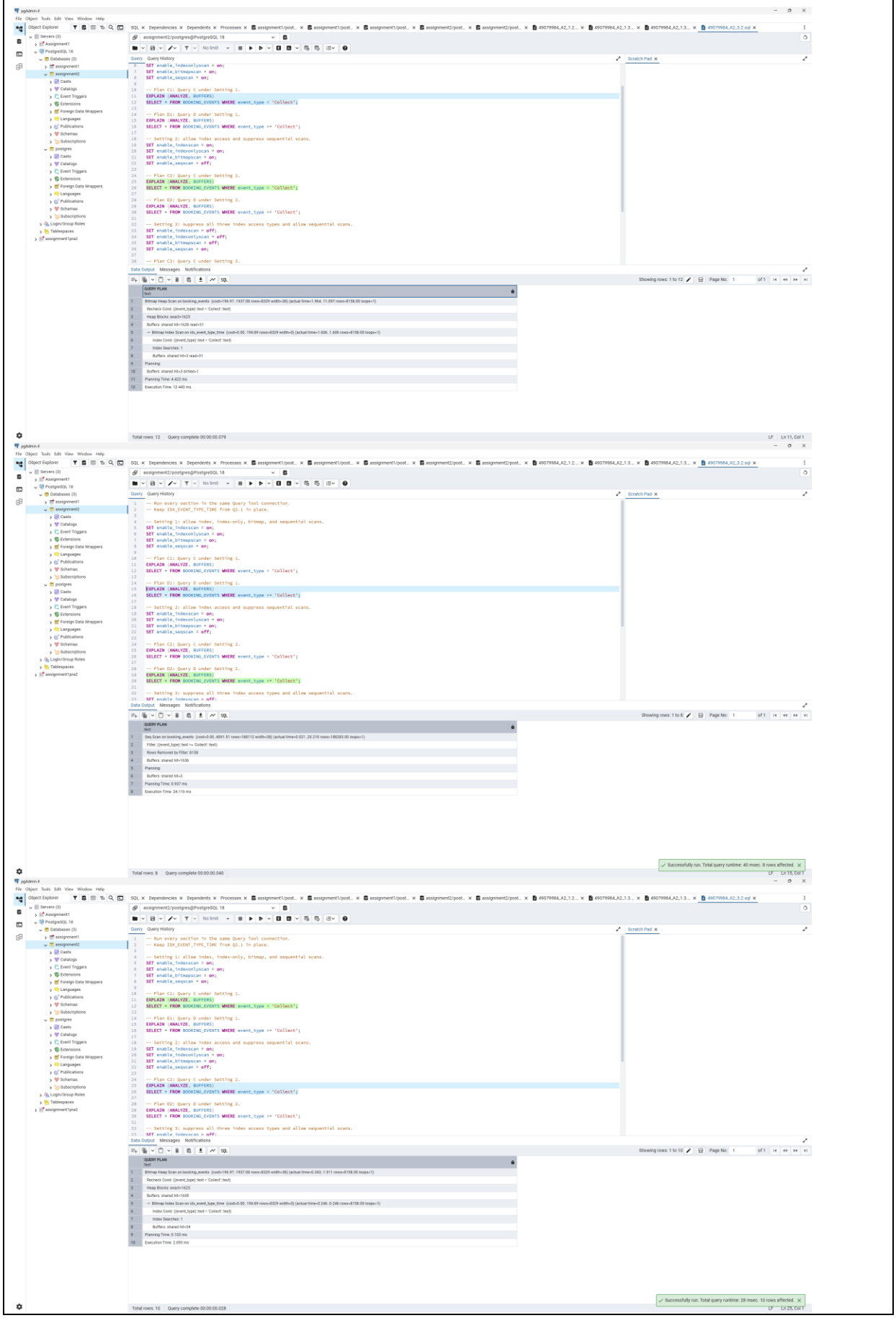
Comments:

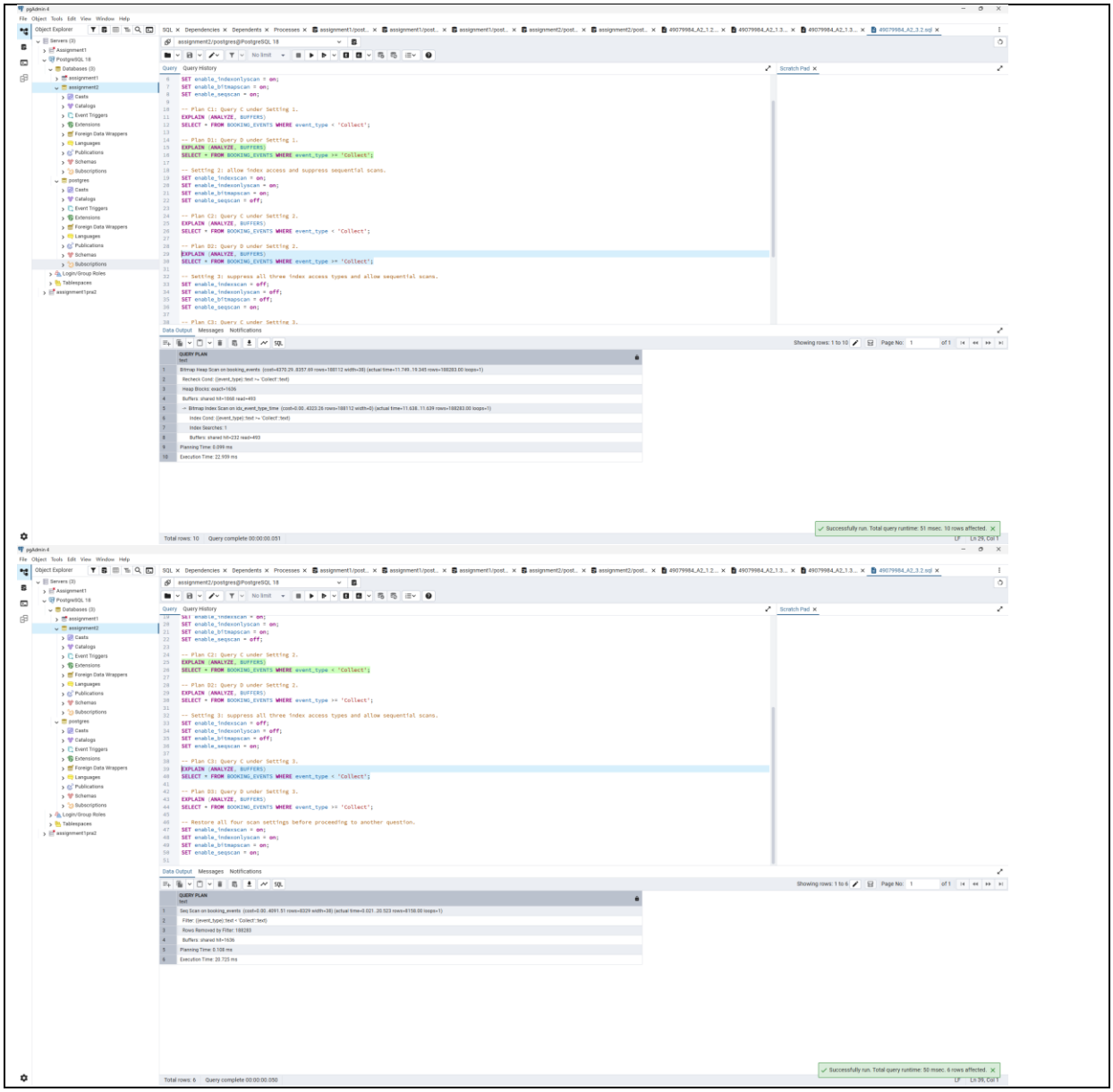
| Screenshot Q3.1 (2) | Marks: /1.5 |
|---|---|

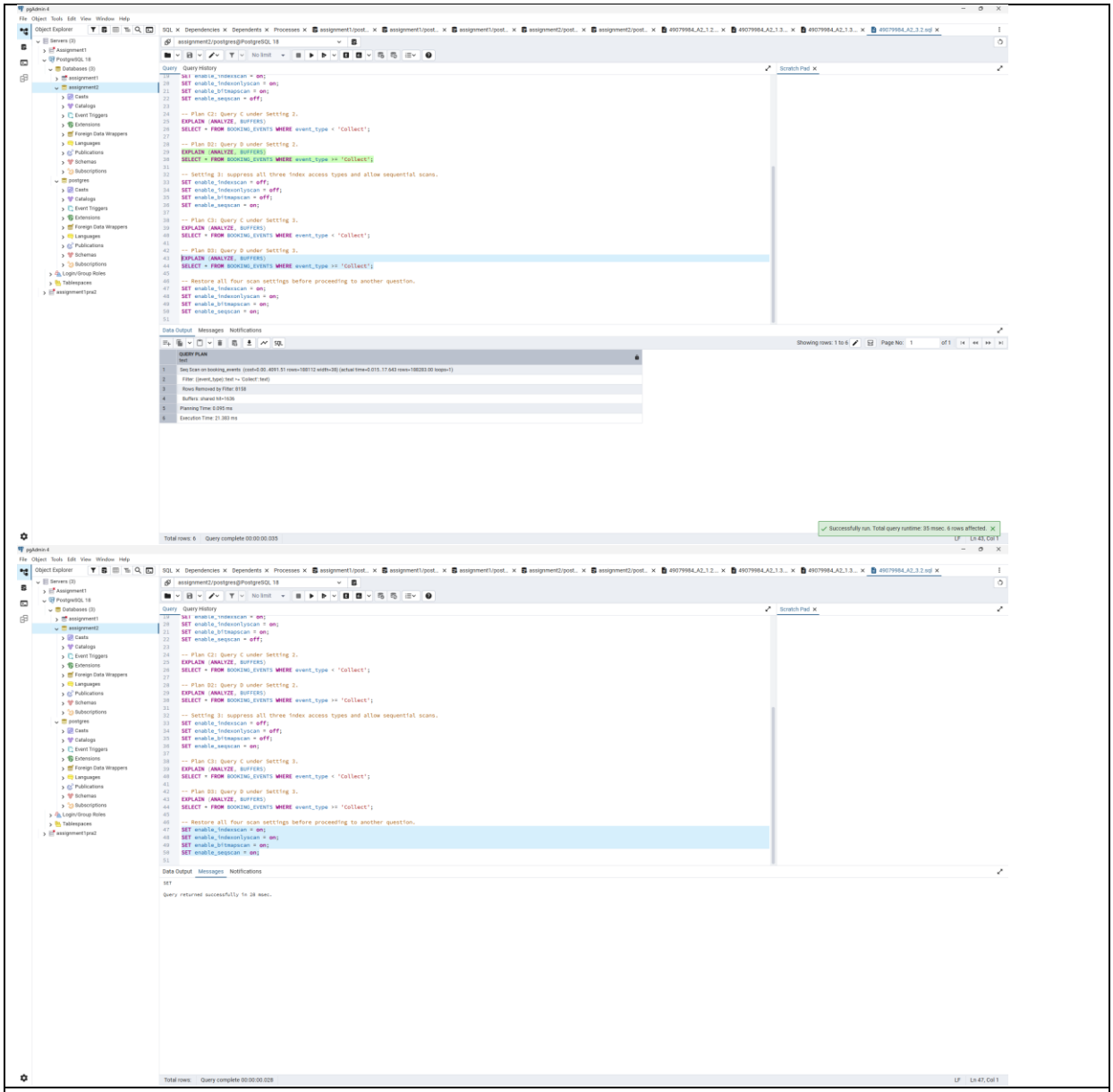


Comments:

| Screenshot Q3.2 (1) | Marks: /2 |
|---|---|







Comments:

| Screenshot Q3.2 (2) | Marks: /2 |
|---|---|

Queries C and D partition 196,441 events at this stage. C returns 8,158 rows (4.15 percent), while D returns 188,283 (95.85 percent). The opposite inequalities on the same indexed column therefore have very different selectivity.

C1 uses Bitmap Index Scan on IDX_EVENT_TYPE_TIME followed by Bitmap Heap Scan. Its cost is 196. 97..1937.08, with 8,329 rows estimated and 8,158 returned. Planning takes 4.423 ms and execution 12.443 ms. It visits 1,625 exact heap blocks; total buffers are 1,628 shared hits and 31 reads. Selective index access costs less than scanning and filtering the whole event log, even though the matching events are spread across many heap pages.

D1 uses Seq Scan, with cost 0. 00..4091.51, 188,112 estimated rows and 188,283 actual rows. Planning takes 0.937 ms and execution 24.116 ms. It reads the heap through 1,636 shared buffer hits and removes 8,158 nonmatching rows. Returning almost the whole table makes a sequential pass cheaper in the cost model than traversing many index entries and then visiting almost every heap page.

C2 retains the bitmap plan and the same cost and row estimates when sequential scans are suppressed. Planning takes 0.103 ms and execution 2.095 ms. All 1,659 buffer accesses are hits, including 34 for the index. The change from C1 includes a warmer cache; suppressing a scan type did not change the chosen access method, so the faster time cannot be attributed to a new plan.

D2 uses Bitmap Index Scan plus Bitmap Heap Scan because sequential scans are suppressed. Its cost rises to 4370. 29..8357.69. It still returns 188,283 rows versus 188,112 estimated and visits all 1,636 heap blocks, with 1,868 shared hits and 493 reads overall. Planning takes 0.099 ms and execution 22.959 ms. This is slightly faster than D1 in this single run despite the higher estimated cost. It does not establish that bitmap access is generally better for this broad predicate; cache state and measurement variation affect elapsed time.

C3 uses Seq Scan after index, index-only and bitmap scans are suppressed. Its cost is 0. 00..4091.51, with 8,329 rows estimated and 8,158 returned. It discards 188,283 rows and has 1,636 shared hits. Planning takes 0.108 ms and execution 20.725 ms, slower than both bitmap C runs. Removing the selective index option forces a full-table read and filter.

D3 also uses Seq Scan, with the same cost and row estimates as D1. Planning takes 0.095 ms and execution 21.383 ms. It returns 188,283 rows, discards 8,158 and has 1,636 shared hits. This remains the natural access method for a query needing nearly every row.

Planner costs are estimates in cost units, not milliseconds. The comparison shows how row selectivity, page access and extra index work influence the chosen method. The switches discourage scan methods; they are useful for this experiment but should not replace normal planner choices in production. All four switches were restored to on.

| Comments: |
|---|
| |

| Screenshot Q4.1 (1) | Marks: /2 |
|---|---|

```
ssignment2=# \r
查询缓存区重置(已清空).
ssignment2=# \if :{?a2_right}
ssignment2@# \echo This is RIGHT. Run this block in the LEFT window.
忽略\echo命令; 使用\endif或Ctrl-C退出当前\if块
ssignment2@# \else
ssignment2=# \set PROMPT1 'LEFT %/%R%x%# '
EFT assignment2=# \set PROMPT2 'LEFT %/%R%x%# '
EFT assignment2=# \set a2_left true
EFT assignment2=#
EFT assignment2=# SELECT pg_backend_pid() AS session_pid,
EFT assignment2-#        current_database() AS database_name;
 session_pid | database_name
-------------+---------------
        8668 | assignment2
(1 行记录)

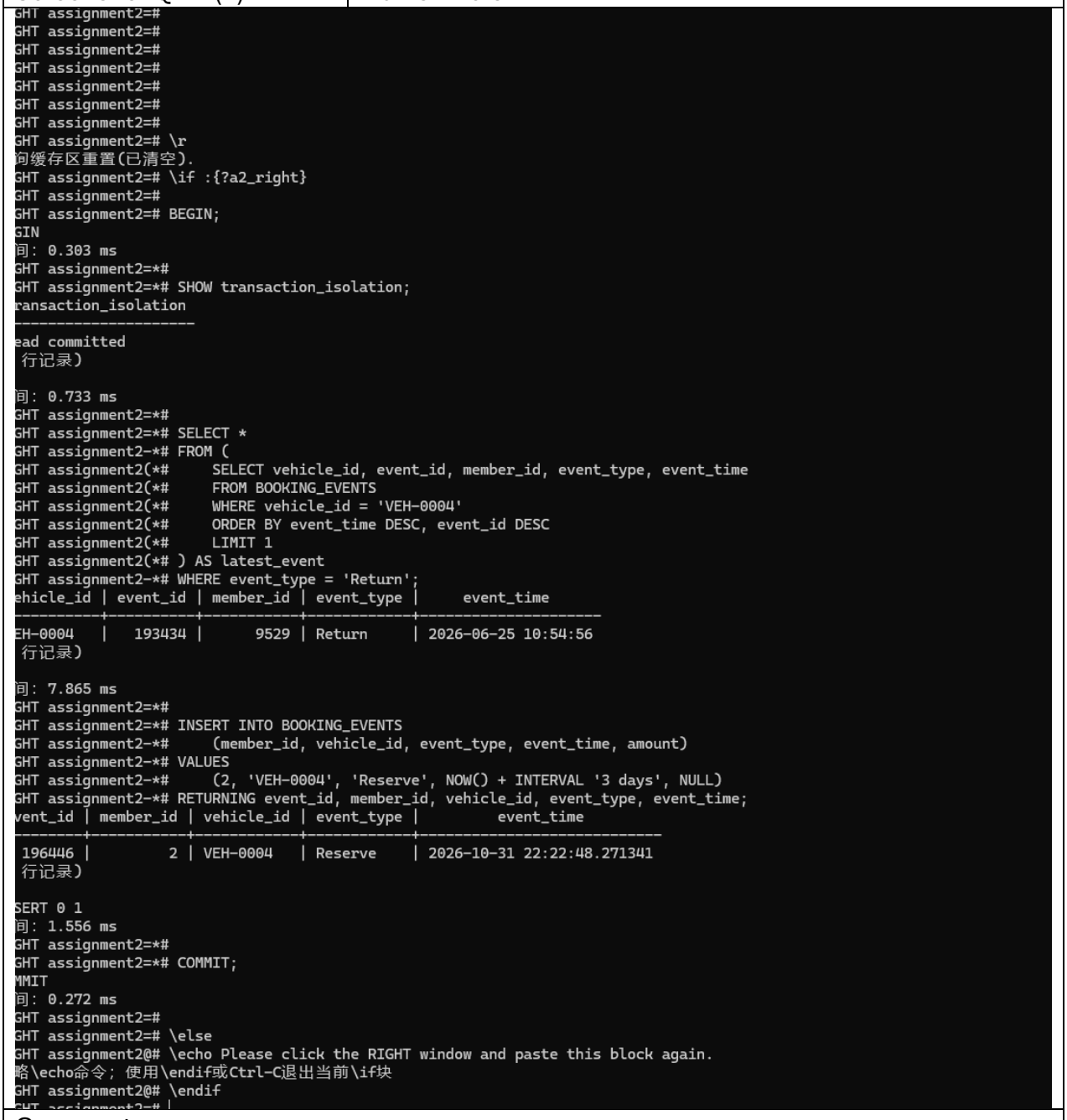
时间: 0.572 ms
EFT assignment2=#
EFT assignment2=# BEGIN;
EGIN
时间: 0.135 ms
EFT assignment2=*# SAVEPOINT collect_attempt;
AVEPOINT
时间: 0.086 ms
EFT assignment2=*#
EFT assignment2=*# SHOW transaction_isolation;
 transaction_isolation
-----------------------
 read committed
(1 行记录)

时间: 0.116 ms
EFT assignment2=*#
EFT assignment2=*# WITH latest_events AS (
EFT assignment2(*#     SELECT DISTINCT ON (vehicle_id)
EFT assignment2(*#         vehicle_id, event_id, member_id, event_type, event_time
EFT assignment2(*#     FROM BOOKING_EVENTS
EFT assignment2(*#     ORDER BY vehicle_id, event_time DESC, event_id DESC
EFT assignment2(*# )
EFT assignment2-*# SELECT vehicle_id, event_id, member_id, event_type, event_time
EFT assignment2-*# FROM latest_events
EFT assignment2-*# WHERE event_type = 'Return'
EFT assignment2-*# ORDER BY vehicle_id
EFT assignment2-*# LIMIT 1;
 vehicle_id | event_id | member_id | event_type |     event_time
------------+----------+-----------+------------+---------------------
 VEH-0004   |   193434 |      9529 | Return     | 2026-06-25 10:54:56
(1 行记录)

时间: 298.961 ms
EFT assignment2=*#
EFT assignment2=*# \endif
```

| Comments: |
|---|

| Screenshot Q4.1 (2) | Marks: /0.5 |
|---|---|



```
GHT assignment2=#
GHT assignment2=#
GHT assignment2=#
GHT assignment2=#
GHT assignment2=#
GHT assignment2=#
GHT assignment2=#
GHT assignment2=# \r
询缓存区重置(已清空).
GHT assignment2=# \if :{?a2_right}
GHT assignment2=#
GHT assignment2=# BEGIN;
GIN
间: 0.303 ms
GHT assignment2=*#
GHT assignment2=*# SHOW transaction_isolation;
ransaction_isolation
---------------------
ead committed
 行记录)

间: 0.733 ms
GHT assignment2=*#
GHT assignment2=*# SELECT *
GHT assignment2-*# FROM (
GHT assignment2(*#     SELECT vehicle_id, event_id, member_id, event_type, event_time
GHT assignment2(*#     FROM BOOKING_EVENTS
GHT assignment2(*#     WHERE vehicle_id = 'VEH-0004'
GHT assignment2(*#     ORDER BY event_time DESC, event_id DESC
GHT assignment2(*#     LIMIT 1
GHT assignment2(*# ) AS latest_event
GHT assignment2-*# WHERE event_type = 'Return';
ehicle_id | event_id | member_id | event_type |     event_time
----------+----------+-----------+------------+---------------------
EH-0004   |   193434 |      9529 | Return     | 2026-06-25 10:54:56
 行记录)

间: 7.865 ms
GHT assignment2=*#
GHT assignment2=*# INSERT INTO BOOKING_EVENTS
GHT assignment2-*#     (member_id, vehicle_id, event_type, event_time, amount)
GHT assignment2-*# VALUES
GHT assignment2-*#     (2, 'VEH-0004', 'Reserve', NOW() + INTERVAL '3 days', NULL)
GHT assignment2-*# RETURNING event_id, member_id, vehicle_id, event_type, event_time;
vent_id | member_id | vehicle_id | event_type |         event_time
--------+-----------+------------+------------+----------------------------
 196446 |         2 | VEH-0004   | Reserve    | 2026-10-31 22:22:48.271341
 行记录)

SERT 0 1
间: 1.556 ms
GHT assignment2=*#
GHT assignment2=*# COMMIT;
MMIT
间: 0.272 ms
GHT assignment2=#
GHT assignment2=# \else
GHT assignment2@# \echo Please click the RIGHT window and paste this block again.
略\echo命令; 使用\endif或Ctrl-C退出当前\if块
GHT assignment2@# \endif
```

| Comments: |
|---|

| Screenshot Q4.1 (3) | Marks: /1 |
|---|---|

```
 行记录)

间: 0.116 ms
FT assignment2=*#
FT assignment2=*# WITH latest_events AS (
FT assignment2(*#     SELECT DISTINCT ON (vehicle_id)
FT assignment2(*#         vehicle_id, event_id, member_id, event_type, event_time
FT assignment2(*#     FROM BOOKING_EVENTS
FT assignment2(*#     ORDER BY vehicle_id, event_time DESC, event_id DESC
FT assignment2(*# )
FT assignment2-*# SELECT vehicle_id, event_id, member_id, event_type, event_time
FT assignment2-*# FROM latest_events
FT assignment2-*# WHERE event_type = 'Return'
FT assignment2-*# ORDER BY vehicle_id
FT assignment2-*# LIMIT 1;
ehicle_id | event_id | member_id | event_type |     event_time
----------+----------+-----------+------------+---------------------
EH-0004   |   193434 |      9529 | Return     | 2026-06-25 10:54:56
 行记录)

间: 298.961 ms
FT assignment2=*#
FT assignment2=*# \endif
FT assignment2=*# \r
询缓存区重置(已清空).
FT assignment2=*# \if :{?a2_left}
FT assignment2=*#
FT assignment2=*# INSERT INTO BOOKING_EVENTS
FT assignment2-*#     (member_id, vehicle_id, event_type, event_time, amount)
FT assignment2-*# VALUES
FT assignment2-*#     (1, 'VEH-0004', 'Collect', NOW(), NULL);
误:  Vehicle VEH-0004 is reserved by member 2
景:  在RAISE的第22行的PL/pgSQL函数udf_bi_collect()
间: 10.551 ms
FT assignment2=!#
FT assignment2=!# COMMIT;
OLLBACK
间: 0.233 ms
FT assignment2=#
FT assignment2=# SELECT
FT assignment2-#     event_id,
FT assignment2-#     member_id,
FT assignment2-#     vehicle_id,
FT assignment2-#     event_type,
FT assignment2-#     event_time,
FT assignment2-#     amount
FT assignment2-# FROM BOOKING_EVENTS
FT assignment2-# WHERE vehicle_id = 'VEH-0004'
FT assignment2-# ORDER BY event_time DESC, event_id DESC
FT assignment2-# LIMIT 3;
vent_id | member_id | vehicle_id | event_type |         event_time         | amount
--------+-----------+------------+------------+----------------------------+--------
 196446 |         2 | VEH-0004   | Reserve    | 2026-10-31 22:22:48.271341 |
 193434 |      9529 | VEH-0004   | Return     | 2026-06-25 10:54:56        |  67.11
 193396 |      9529 | VEH-0004   | Collect    | 2026-06-25 09:40:08        |
 行记录)

间: 8.172 ms
FT assignment2=#
FT assignment2=# \else
FT assignment2@# \echo Please click the LEFT window and paste this block again.
略\echo命令; 使用\endif或Ctrl-C退出当前\if块
FT assignment2@# \endif
FT assignment2=#
FT assignment2=#
```

Comments:

| Screenshot Q4.1 (4) | Marks: /2.5 |
|---|---|

The schedule is T1 BEGIN and availability read; T2 BEGIN and availability read; T2 Reserve and COMMIT; T1 Collect attempt; trigger exception; T1 ROLLBACK. T1 initially sees an available car, but that answer becomes stale before its next operation.

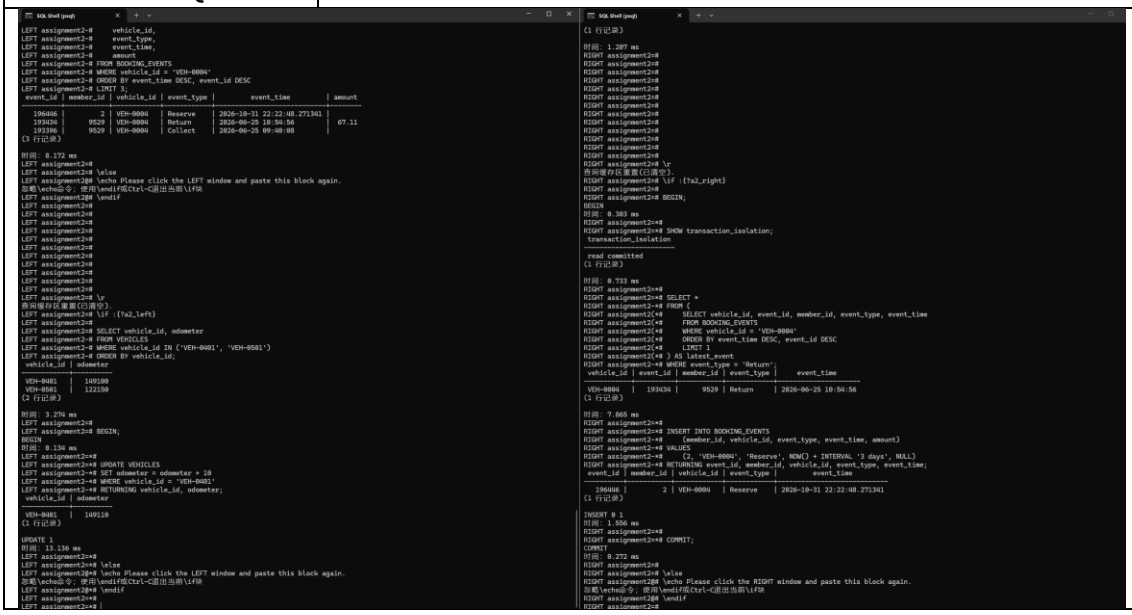
At READ COMMITTED, each statement uses a fresh snapshot. The Collect attempt runs after T2 has committed, so the trigger can see the committed Reserve. The trigger orders events by event time and treats that future-dated reservation as the latest state. Because it belongs to member 2, it refuses a Collect by member 1.

An ordinary SELECT reads without a row lock that reserves this car. Its ACCESS SHARE relation lock is compatible with the other transaction's insertion, so it does not prevent T2 from changing the answer. T1 stays open but does not retain its first statement's snapshot for later statements. In this schedule the rejection is a business-rule exception, with no circular lock wait.

For the co-operative, the person at the parking bay is refused the car even though the other person reserved it three days ahead. The simplified trigger makes a future reservation an immediate hold. Coordinating availability checks and writes would address races, while interval-aware booking rules would also be needed to distinguish available now from reserved later.

Comments:

| Screenshot Q4.2 | Marks: /3 |
|---|---|

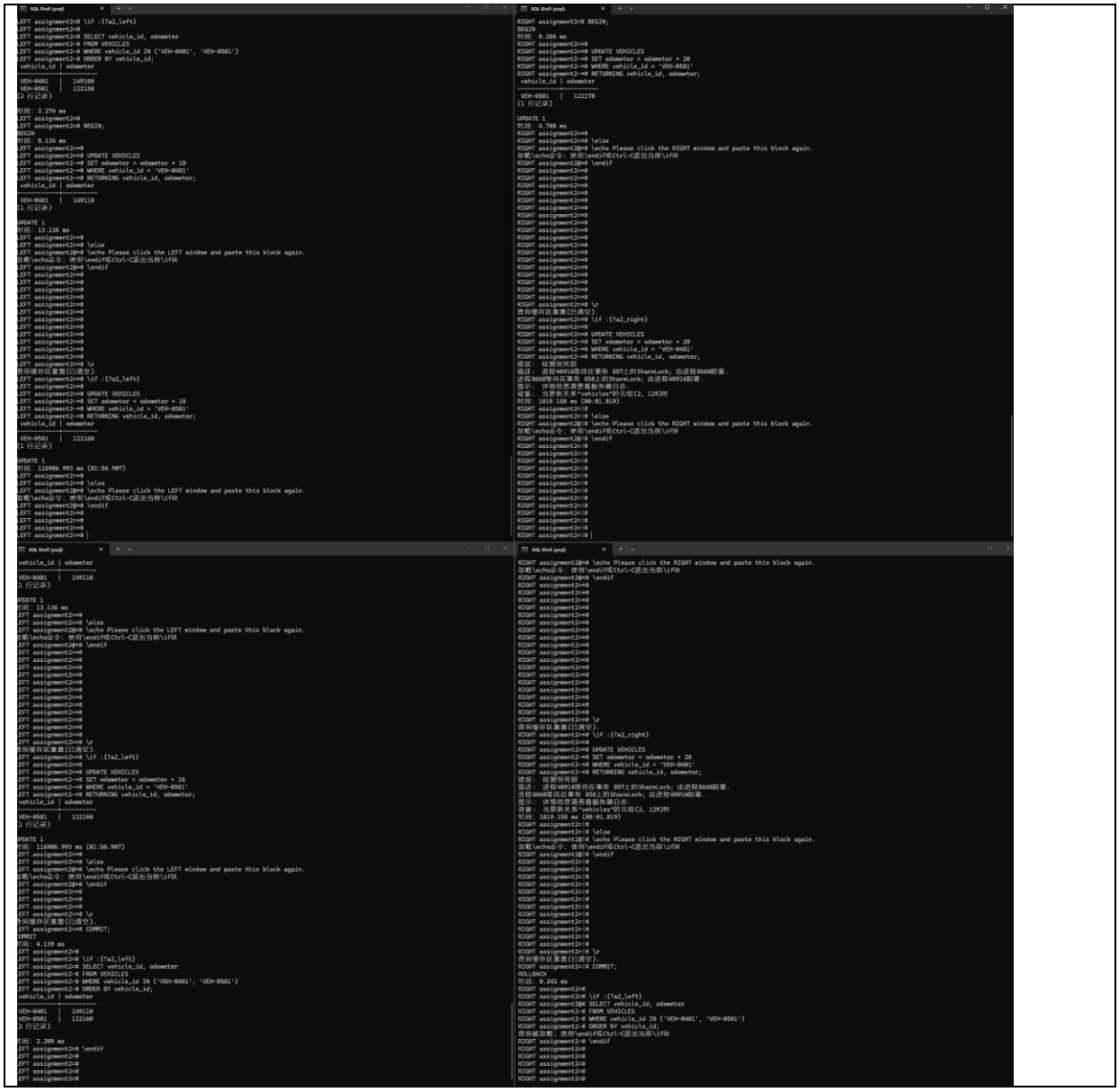


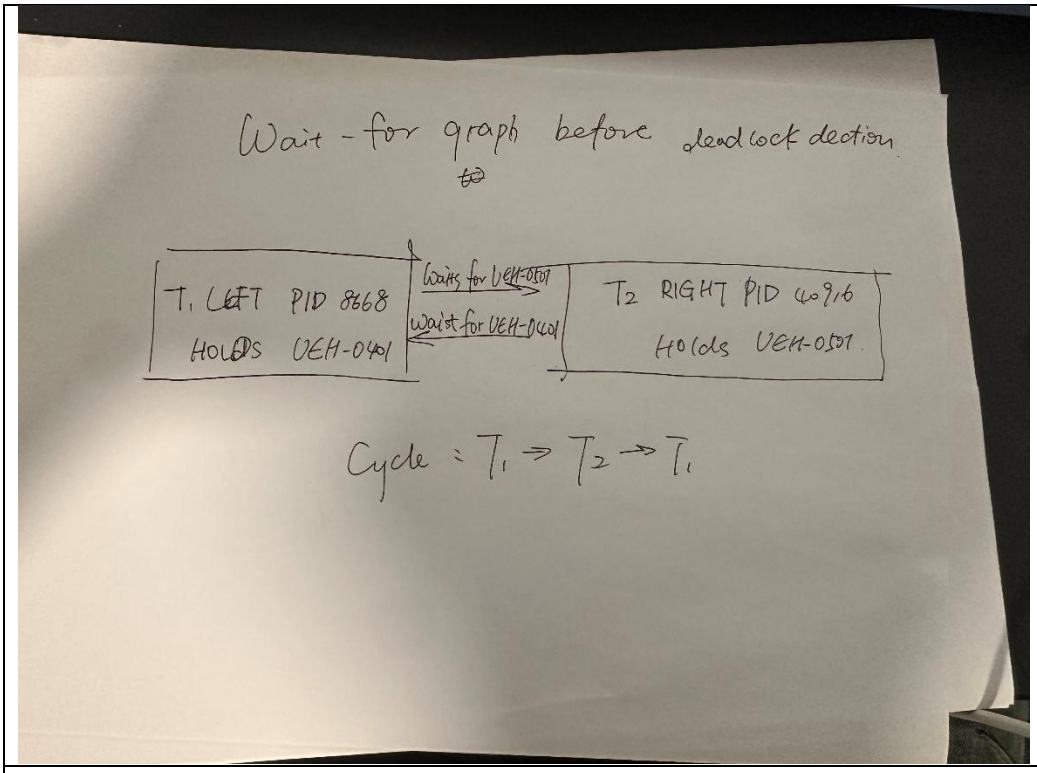
```
RIGHT assignment2=#
RIGHT assignment2=#
RIGHT assignment2=#
RIGHT assignment2=#
RIGHT assignment2=#
RIGHT assignment2=# \r
查询缓存区重置(已清空).
RIGHT assignment2=# \if :{?a2_right}
RIGHT assignment2=#
RIGHT assignment2=# BEGIN;
BEGIN
时间: 0.206 ms
RIGHT assignment2=*#
RIGHT assignment2=*# UPDATE VEHICLES
RIGHT assignment2-*# SET odometer = odometer + 20
RIGHT assignment2-*# WHERE vehicle_id = 'VEH-0501'
RIGHT assignment2-*# RETURNING vehicle_id, odometer;
 vehicle_id | odometer
------------+----------
 VEH-0501   |   122170
(1 行记录)

UPDATE 1
时间: 4.708 ms
RIGHT assignment2=*#
RIGHT assignment2=*# \else
RIGHT assignment2@*# \echo Please click the RIGHT window and paste this block again.
忽略\echo命令; 使用\endif或Ctrl-C退出当前\if块
RIGHT assignment2@*# \endif
RIGHT assignment2=*#
RIGHT assignment2=*#
RIGHT assignment2=*#
RIGHT assignment2=*#
RIGHT assignment2=*#
RIGHT assignment2=*#
RIGHT assignment2=*#
RIGHT assignment2=*#
 VEH-0501   |   122150
(2 行记录)

时间: 3.274 ms
LEFT assignment2=#
LEFT assignment2=# BEGIN;
BEGIN
时间: 0.134 ms
LEFT assignment2=*#
LEFT assignment2=*# UPDATE VEHICLES
LEFT assignment2-*# SET odometer = odometer + 10
LEFT assignment2-*# WHERE vehicle_id = 'VEH-0401'
LEFT assignment2-*# RETURNING vehicle_id, odometer;
 vehicle_id | odometer
------------+----------
 VEH-0401   |   149110
(1 行记录)

UPDATE 1
时间: 13.136 ms
LEFT assignment2=*#
LEFT assignment2=*# \else
LEFT assignment2@*# \echo Please click the LEFT window and paste this block again.
忽略\echo命令; 使用\endif或Ctrl-C退出当前\if块
LEFT assignment2@*# \endif
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*# \r
查询缓存区重置(已清空).
LEFT assignment2=*# \if :{?a2_left}
LEFT assignment2=*#
LEFT assignment2=*# UPDATE VEHICLES
LEFT assignment2-*# SET odometer = odometer + 10
LEFT assignment2-*# WHERE vehicle_id = 'VEH-0501'
LEFT assignment2-*# RETURNING vehicle_id, odometer;
```


SQL Shell (psql)
LEFT assignment2=# \if :{?a2_left}
LEFT assignment2=# SELECT vehicle_id, odometer
LEFT assignment2-# FROM VEHICLES
LEFT assignment2-# WHERE vehicle_id IN ('VEH-0401', 'VEH-0501')
LEFT assignment2-# ORDER BY vehicle_id;
vehicle_id | odometer
VEH-0401 | 149100
VEH-0501 | 122150
(2 行记录)
时间: 3.274 ms
LEFT assignment2=#
LEFT assignment2=# BEGIN;
BEGIN
时间: 0.134 ms
LEFT assignment2=#
LEFT assignment2=# UPDATE VEHICLES
LEFT assignment2-# SET odometer = odometer + 10
LEFT assignment2-# WHERE vehicle_id = 'VEH-0401'
LEFT assignment2-# RETURNING vehicle_id, odometer;
vehicle_id | odometer
VEH-0401 | 149110
(1 行记录)
UPDATE 1
时间: 13.136 ms
LEFT assignment2=#
LEFT assignment2=# \else
LEFT assignment2@=# \echo Please click the LEFT window and paste this block again.
忽略\echo命令: 使用\endif或Ctrl-C退出当前\if块
LEFT assignment2@=# \endif
LEFT assignment2=#
LEFT assignment2=# \r
查询缓存区重置(已清空).
LEFT assignment2=# \if :{?a2_left}
LEFT assignment2=#
LEFT assignment2=# UPDATE VEHICLES
LEFT assignment2-# SET odometer = odometer + 10
LEFT assignment2-# WHERE vehicle_id = 'VEH-0501'
LEFT assignment2-# RETURNING vehicle_id, odometer;
vehicle_id | odometer
VEH-0501 | 122160
(1 行记录)
UPDATE 1
时间: 116986.993 ms (01:56.987)
LEFT assignment2=#
LEFT assignment2=# \else
LEFT assignment2@=# \echo Please click the LEFT window and paste this block again.
忽略\echo命令: 使用\endif或Ctrl-C退出当前\if块
LEFT assignment2@=# \endif
LEFT assignment2=#
LEFT assignment2=#
LEFT assignment2=#
RIGHT assignment2=# BEGIN;
BEGIN
时间: 0.286 ms
RIGHT assignment2=#
RIGHT assignment2=# UPDATE VEHICLES
RIGHT assignment2-# SET odometer = odometer + 20
RIGHT assignment2-# WHERE vehicle_id = 'VEH-0501'
RIGHT assignment2-# RETURNING vehicle_id, odometer;
vehicle_id | odometer
VEH-0501 | 122170
(1 行记录)
UPDATE 1
时间: 4.708 ms
RIGHT assignment2=#
RIGHT assignment2=# \else
RIGHT assignment2@=# \echo Please click the RIGHT window and paste this block again.
忽略\echo命令: 使用\endif或Ctrl-C退出当前\if块
RIGHT assignment2@=# \endif
RIGHT assignment2=#
RIGHT assignment2=# \r
查询缓存区重置(已清空).
RIGHT assignment2=# \if :{?a2_right}
RIGHT assignment2=#
RIGHT assignment2=# UPDATE VEHICLES
RIGHT assignment2-# SET odometer = odometer + 20
RIGHT assignment2-# WHERE vehicle_id = 'VEH-0401'
RIGHT assignment2-# RETURNING vehicle_id, odometer;
错误: 检测到死锁
详述: 进程40916等待在事务 857上的ShareLock; 由进程8668阻塞.
进程8668等待在事务 858上的ShareLock; 由进程40916阻塞.
提示: 详细信息请查看服务器日志.
背景: 当更新关系"vehicles"的元组(2, 129)时
时间: 1819.158 ms (00:01.819)
RIGHT assignment2=#
RIGHT assignment2=# \else
RIGHT assignment2@=# \echo Please click the RIGHT window and paste this block again.
忽略\echo命令: 使用\endif或Ctrl-C退出当前\if块
RIGHT assignment2@=# \endif
RIGHT assignment2=#
RIGHT assignment2=# \r
查询缓存区重置(已清空).
RIGHT assignment2=# \if :{?a2_right}
RIGHT assignment2=#
RIGHT assignment2=# UPDATE VEHICLES
RIGHT assignment2-# SET odometer = odometer + 20
RIGHT assignment2-# WHERE vehicle_id = 'VEH-0401'
RIGHT assignment2-# RETURNING vehicle_id, odometer;
错误: 检测到死锁
详述: 进程40916等待在事务 857上的ShareLock; 由进程8668阻塞.
进程8668等待在事务 858上的ShareLock; 由进程40916阻塞.
提示: 详细信息请查看服务器日志.
背景: 当更新关系"vehicles"的元组(2, 129)时
时间: 1819.158 ms (00:01.819)
RIGHT assignment2=#
RIGHT assignment2=# \else
RIGHT assignment2@=# \echo Please click the RIGHT window and paste this block again.
忽略\echo命令: 使用\endif或Ctrl-C退出当前\if块
RIGHT assignment2@=# \endif
RIGHT assignment2=#
RIGHT assignment2=# \r
查询缓存区重置(已清空).
RIGHT assignment2=# COMMIT;
ROLLBACK
时间: 0.242 ms
RIGHT assignment2=#
RIGHT assignment2=# \if :{?a2_left}
RIGHT assignment2@=# SELECT vehicle_id, odometer
RIGHT assignment2-# FROM VEHICLES
RIGHT assignment2-# WHERE vehicle_id IN ('VEH-0401', 'VEH-0501')
RIGHT assignment2-# ORDER BY vehicle_id;
查询被忽略: 使用\endif或Ctrl-C退出当前\if块
RIGHT assignment2-# \endif
RIGHT assignment2=#




Comments:

| Screenshot P | Marks: /2 |
|---|---|
| | |

Comments:

**Penalty:** _____

Reason of the Penalty

(Late Submission/No Show to Tutor/Purely AI Generated Solutions): _____

**Final Marks :** _____ / 30

---ooo000OOO000ooo---